

Core Language Working Group "ready" Issues for the March, 2024 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4971](#).

453. References may only bind to “valid” objects

Section: 9.3.4.3 [\[dcl.ref\]](#) **Status:** ready **Submitter:** Gennaro Prota **Date:** 18 Jan 2004

9.3.4.3 [\[dcl.ref\]](#) paragraph 4 says:

A reference shall be initialized to refer to a valid object or function. [Note: in particular, a null reference cannot exist in a well-defined program, because the only way to create such a reference would be to bind it to the "object" obtained by dereferencing a null pointer, which causes undefined behavior ...]

What is a "valid" object? In particular the expression "valid object" seems to exclude uninitialized objects, but the response to Core Issue 363 clearly says that's not the intent. This is an example (overloading construction on constness of *this) by John Potter, which I think is supposed to be legal C++ though it binds references to objects that are not initialized yet:

```
struct Fun {
    int x, y;
    Fun (int x, Fun const&) : x(x), y(42) { }
    Fun (int x, Fun&) : x(x), y(0) { }
};
int main () {
    const Fun f1 (13, f1);
    Fun f2 (13, f2);
    cout << f1.y << " " << f2.y << "\n";
}
```

Suggested resolution: Changing the final part of 9.3.4.3 [\[dcl.ref\]](#) paragraph 4 to:

A reference shall be initialized to refer to an object or function. From its point of declaration on (see 6.4.2 [\[basic.scope.pdecl\]](#)) its name is an lvalue which refers to that object or function. The reference may be initialized to refer to an uninitialized object but, in that case, it is usable in limited ways (6.7.3 [\[basic.life\]](#), paragraph 6) [Note: On the other hand, a declaration like this:

```
int & ref = *(int*)0;
```

is ill-formed because ref will not refer to any object or function]

I also think a "No diagnostic is required." would better be added (what about something like `int& r = r; ?`)

Proposed Resolution (October, 2004) [SUPERSEDED]:

(Note: the following wording depends on the proposed resolution for issue 232.)

Change 9.3.4.3 [\[dcl.ref\]](#) paragraph 4 as follows:

~~A reference shall be initialized to refer to a valid object or function. If an lvalue to which a reference is directly bound designates neither an existing object or function of an appropriate type (9.4.4 [\[dcl.init.ref\]](#)), nor a region of memory of suitable size and alignment to contain an object of the reference's type (6.7.2 [\[intro.object\]](#), 6.7.3 [\[basic.life\]](#), 6.8 [\[basic.types\]](#)), the behavior is undefined.~~ [Note: in particular, a null reference cannot exist in a well-defined program, because the only way to create such a reference would be to bind it to the "object" empty lvalue obtained by dereferencing a null pointer, which ~~causes undefined behavior.~~ ~~As does not designate an object or function.~~ Also, as described in 11.14.10 [\[class.bit\]](#), a reference cannot be bound directly to a bit-field.]

The name of a reference shall not be used in its own initializer. Any other use of a reference before it is initialized results in undefined behavior. [Example:

```
int& f(int&);
int& g();

extern int& ir3;
int* ip = 0;

int& ir1 = *ip;    // undefined behavior: null pointer
int& ir2 = f(ir3); // undefined behavior: ir3 not yet initialized
int& ir3 = g();
int& ir4 = f(ir4); // ill-formed: ir4 used in its own initializer
```

—end example]

Rationale: The proposed wording goes beyond the specific concerns of the issue. It was noted that, while the current wording makes cases like `int& r = r;` ill-formed (because `r` in the initializer does not "refer to a valid object"), an inappropriate initialization can only be detected, if at all, at runtime and thus "undefined behavior" is a more appropriate treatment. Nevertheless, it was deemed desirable to continue to require a diagnostic for obvious compile-time cases.

It was also noted that the current Standard does not say anything about using a reference before it is initialized. It seemed reasonable to address both of these concerns in the same wording proposed to resolve this issue.

Notes from the April, 2005 meeting:

The CWG decided that whether to require an implementation to diagnose initialization of a reference to itself should be handled as a separate issue (504) and also suggested referring to "storage" instead of "memory" (because 6.7.2 [intro.object] defines an object as a "region of storage").

Proposed Resolution (April, 2005) [SUPERSEDED]:

(Note: the following wording depends on the proposed resolution for issue 232.)

Change 9.3.4.3 [dcl.ref] paragraph 4 as follows:

~~A reference shall be initialized to refer to a valid object or function.~~ **If an lvalue to which a reference is directly bound designates neither an existing object or function of an appropriate type (9.4.4 [dcl.init.ref]), nor a region of storage of suitable size and alignment to contain an object of the reference's type (6.7.2 [intro.object], 6.7.3 [basic.life], 6.8 [basic.types]), the behavior is undefined.** [Note: in particular, a null reference cannot exist in a well-defined program, because the only way to create such a reference would be to bind it to the "object" empty lvalue obtained by dereferencing a null pointer, which ~~causes undefined behavior.~~ **As does not designate an object or function. Also, as** described in 11.4.10 [class.bit], a reference cannot be bound directly to a bit-field.]

Any use of a reference before it is initialized results in undefined behavior. [Example:

```
int& f(int&);
int& g();

extern int& ir3;
int* ip = 0;

int& ir1 = *ip;      // undefined behavior: null pointer
int& ir2 = f(ir3);   // undefined behavior: ir3 not yet initialized
int& ir3 = g();
int& ir4 = f(ir4);   // undefined behavior: ir4 used in its own initializer
```

—end example]

Note (February, 2006):

The word "use" in the last paragraph of the proposed resolution was intended to refer to the description in 6.3 [basic.def.odr] paragraph 2. However, that section does not define what it means for a reference to be "used," dealing only with objects and functions. Additional drafting is required to extend 6.3 [basic.def.odr] paragraph 2 to apply to references.

Additional note (May, 2008):

The proposed resolution for issue 570 adds wording to define "use" for references.

Note, January, 2012:

The resolution should also probably deal with the fact that the "one-past-the-end" address of an array does not designate a valid object (even if such a pointer might "point to" an object of the correct type, per 6.8.4 [basic.compound]) and thus is not suitable for the lvalue-to-rvalue conversion.

CWG 2023-11-06

We need a (possibly out-of-lifetime) object, not just a region of storage here. Empty lvalues do not exist. Otherwise, the direction is confirmed.

Proposed resolution (approved by CWG 2023-11-10) [SUPERSEDED]:

1. Change in 7.2.1 [basic.lval] paragraph 11 as follows:

An object of dynamic type T_{obj} is *type-accessible* through a glvalue of type T_{ref} if T_{ref} is similar (7.3.6 [conv.qual]) to:

- T_{obj} ,
- a type that is the signed or unsigned type corresponding to T_{obj} , or
- a `char`, `unsigned char`, or `std::byte` type.

If a program attempts to access (3.1 [defs.access]) the stored value of an object through a glvalue ~~whose type is not similar (7.3.6 [conv.qual]) to one of the following types~~ **through which it is not type-accessible**, the behavior is undefined. [Footnote: ...]

- the dynamic type of the object,
- a type that is the signed or unsigned type corresponding to the dynamic type of the object, or
- a `char`, `unsigned char`, or `std::byte` type.

If a program invokes a defaulted copy/move constructor or copy/move assignment operator for a union of type U with a glvalue argument that does not denote an object of type `cv U` within its lifetime, the behavior is undefined.

2. Change in 7.6.1.3 [expr.call] paragraph 5 as follows:

A type T_{call} is call-compatible with a function type T_{func} if T_{call} is the same type as T_{func} or if the type "pointer to T_{func} " can be converted to type "pointer to T_{call} " via a function pointer conversion (7.3.14 [conv.fctptr]). Calling a function through an expression whose function type E is different from the function type F of the called function's definition results in undefined behavior unless the type "pointer to F " can be converted to the type "pointer to E " via a function pointer conversion (7.3.14 [conv.fctptr]). [Note 4: The exception applies. This requirement allows the case when the expression has the type of a potentially-throwing function, but the called function has a non-throwing exception specification, and the function types are otherwise the same. —end note]

3. Change and split in 9.3.4.3 [dcl.ref] paragraph 5 as follows:

There shall be no references to references, no arrays of references, and no pointers to references. The declaration of a reference shall contain an initializer (9.4.4 [dcl.init.ref]) except when the declaration contains an explicit extern specifier (9.2.2 [dcl.stc]), is a class member (11.4 [class.mem]) declaration within a class definition, or is the declaration of a parameter or a return type (9.3.4.6 [dcl.fct]); see 6.2 [basic.def]. ~~A reference shall be initialized to refer to a valid object or function.~~

Attempting to bind a reference to a function where the initializer is a glvalue whose type is not call-compatible (7.6.1.3 [expr.call]) with the type of the function's definition results in undefined behavior. Attempting to bind a reference to an object where the initializer is a glvalue through which the object is not type-accessible (7.2.1 [basic.lval]) results in undefined behavior. [Note 2: In particular, a null reference cannot exist in a well-defined program, because the only way to create such a reference would be to bind it to the "object" obtained by indirection through a null pointer, which causes undefined behavior. The object designated by such a glvalue can be outside its lifetime (6.7.3 [basic.life]). Because a null pointer value or a pointer past the end of an object does not point to an object, a reference in a well-defined program cannot refer to such things; see 7.6.2.2 [expr.unary.op]. As described in 11.4.10 [class.bit], a reference cannot be bound directly to a bit-field. —end note] An odr-use (6.3 [basic.def.odr]) of a reference that does not happen after (6.9.2.2 [intro.races]) its initialization results in undefined behavior. [Example:

```
int &f(int&);
int &g();
extern int &ir3;
int *ip = 0;
int &ir1 = *ip;    // undefined behavior: null pointer
int &ir2 = f(ir3); // undefined behavior: ir3 not yet initialized
int &ir3 = g();
int &ir4 = f(ir4); // undefined behavior: ir4 used in its own initializer
```

-- end example]

Additional notes (November, 2023):

An odr-use is a property of the program, not an evaluation that participates in the "happens before" relation.

Proposed resolution (approved by CWG 2024-03-20):

1. Change in 7.2.1 [basic.lval] paragraph 11 as follows:

An object of dynamic type T_{obj} is type-accessible through a glvalue of type T_{ref} if T_{ref} is similar (7.3.6 [conv.qual]) to:

- T_{obj} ,
- a type that is the signed or unsigned type corresponding to T_{obj} , or
- a char, unsigned char, or std::byte type.

If a program attempts to access (3.1 [defs.access]) the stored value of an object through a glvalue whose type is not similar (7.3.6 [conv.qual]) to one of the following types through which it is not type-accessible, the behavior is undefined. [Footnote: ...]

- the dynamic type of the object,
- a type that is the signed or unsigned type corresponding to the dynamic type of the object, or
- a char, unsigned char, or std::byte type.

If a program invokes a defaulted copy/move constructor or copy/move assignment operator for a union of type U with a glvalue argument that does not denote an object of type cv U within its lifetime, the behavior is undefined.

2. Change in 7.6.1.3 [expr.call] paragraph 5 as follows:

A type T_{call} is call-compatible with a function type T_{func} if T_{call} is the same type as T_{func} or if the type "pointer to T_{func} " can be converted to type "pointer to T_{call} " via a function pointer conversion (7.3.14 [conv.fctptr]). Calling a function through an expression whose function type E is different from the function type F of the called function's definition results in undefined behavior unless the type "pointer to F " can be converted to the type "pointer to E " via a function pointer conversion (7.3.14 [conv.fctptr]). [Note 4: The exception applies. This requirement allows the case when the expression has the type of a potentially-throwing function, but the called function has a non-throwing exception specification, and the function types are otherwise the same. —end note]

3. Change and split in 9.3.4.3 [dcl.ref] paragraph 5 as follows:

There shall be no references to references, no arrays of references, and no pointers to references. The declaration of a reference shall contain an initializer (9.4.4 [dcl.init.ref]) except when the declaration contains an explicit extern specifier (9.2.2 [dcl.stc]), is a class member (11.4 [class.mem]) declaration within a class definition, or is the declaration of a parameter or a return type (9.3.4.6 [dcl.fct]); see 6.2 [basic.def]. ~~A reference shall be initialized to refer to a valid object or function.~~

Attempting to bind a reference to a function where the converted initializer is a glvalue whose type is not call-compatible (7.6.1.3 [expr.call]) with the type of the function's definition results in undefined behavior. Attempting to bind a reference to an object where the converted initializer is a glvalue through which the object is not type-accessible (7.2.1 [basic.lval]) results in undefined behavior. [Note 2: In particular, a null reference cannot exist in a well-defined program, because the only way to create such a reference would be to bind it to the "object" obtained by indirection through a null pointer, which causes undefined behavior. The object designated by such a glvalue can be outside its lifetime (6.7.3 [basic.life]). Because a null pointer value or a pointer past the end of an object does not point to an

object, a reference in a well-defined program cannot refer to such things; see 7.6.2.2 [expr.unary.op]. As described in 11.4.10 [class.bit], a reference cannot be bound directly to a bit-field. —end note] The behavior of an evaluation of a reference (7.5.4 [expr.prim.id], 7.6.1.5 [expr.ref]) that does not happen after (6.9.2.2 [intro.races]) the initialization of the reference is undefined. [Example:

```
int &f(int&);
int &g();
extern int &ir3;
int *ip = 0;
int &ir1 = *ip;    // undefined behavior: null pointer
int &ir2 = f(ir3); // undefined behavior: ir3 not yet initialized
int &ir3 = g();
int &ir4 = f(ir4); // undefined behavior: ir4 used in its own initializer

char x alignas(int);
int &ir5 = reinterpret_cast<int *>(&x); // undefined behavior: initializer refers to char object

-- end example ]
```

1954. typeid null dereference check in subexpressions

Section: 7.6.1.8 [expr.typeid] **Status:** ready **Submitter:** David Majnemer **Date:** 2014-06-23

According to 7.6.1.8 [expr.typeid] paragraph 2,

If the glvalue expression is obtained by applying the unary * operator to a pointer⁶⁹ and the pointer is a null pointer value (7.3.12 [conv.ptr]), the typeid expression throws an exception (14.2 [except.throw]) of a type that would match a handler of type std::bad_typeid exception (17.7.5 [bad.typeid]).

The footnote makes clear that this requirement applies without regard to parentheses, but it is unspecified whether it applies when the dereference occurs in a subexpression of the operand (e.g., in the second operand of the comma operator or the second or third operand of a conditional operator). There is implementation divergence on this question.

Proposed resolution (approved by CWG 2023-11-09):

Insert a new paragraph before 7.6.1.8 [expr.typeid] paragraph 3 and change the latter as follows:

If an *expression* operand of typeid is a possibly-parenthesized *unary-expression* whose *unary-operator* is * and whose operand evaluates to a null pointer value (6.8.4 [basic.compound]), the typeid expression throws an exception (14.2 [except.throw]) of a type that would match a handler of type std::bad_typeid (17.7.5 [bad.typeid]). [Note: In other contexts, evaluating such a *unary-expression* results in undefined behavior (7.6.2.2 [expr.unary.op]) -- end note]

When typeid is applied to a glvalue whose type is a polymorphic class type (11.7.3 [class.virtual]), the result refers to a std::type_info object representing the type of the most derived object (6.7.2 [intro.object]) (that is, the dynamic type) to which the glvalue refers. If the glvalue is obtained by applying the unary * operator to a pointer [Footnote: ...] and the pointer is a null pointer value (6.8.4 [basic.compound]), the typeid expression throws an exception (14.2 [except.throw]) of a type that would match a handler of type std::bad_typeid exception (17.7.5 [bad.typeid]).

2476. placeholder-type-specifiers and function declarators

Section: 9.2.9.7.1 [dcl.spec.auto.general] **Status:** ready **Submitter:** Davis Herring **Date:** 2021-01-29

According to 9.2.9.7.1 [dcl.spec.auto.general] paragraph 3,

The placeholder type can appear with a function declarator in the *decl-specifier-seq*, *type-specifier-seq*, *conversion-function-id*, or *trailing-return-type*, in any context where such a declarator is valid. If the function declarator includes a *trailing-return-type* (9.3.4.6 [dcl.fct]), that *trailing-return-type* specifies the declared return type of the function. Otherwise, the function declarator shall declare a function.

This wording disallows a declaration like

```
int f();
auto (*fp)()=f;
```

The requirement to declare a function was introduced by the resolution of issue 1892.

Proposed resolution (April, 2021) [SUPERSEDED]:

Change 9.2.9.7.1 [dcl.spec.auto.general] paragraph 3 as follows:

The placeholder type can appear with a function declarator in the *decl-specifier-seq*, *type-specifier-seq*, *conversion-function-id*, or *trailing-return-type*, in any context where such a declarator is valid if the function declarator includes a *trailing-return-type* T (9.3.4.6 [dcl.fct]) or declares a function. If the function declarator includes a *trailing-return-type* (9.3.4.6 [dcl.fct]), that *trailing-return-type* specifies In the former case, T is the declared return type of the function. Otherwise, the function declarator shall declare a function. If the declared return type of the a function contains a placeholder type, the return type of the function is deduced from non-discarded return statements, if any, in the body of the function (8.5.2 [stmt.if]).

Additional notes (May, 2021):

It was observed that the proposed resolution above does not address the example in the issue, since `fp` neither has a *trailing-return-type* nor declares a function. Presumably another case in which a function declarator with a placeholder return type should be permitted is in the declaration of a variable in which the type is deduced from its initializer.

It was also noted in passing that the deduction in the example is only partial: the parameter-type-list is specified by the declarator and only the return type is deduced from the initializer. Although this example is supported by current implementations, there is implementation divergence in the support of another case in which only part of the variable's type is deduced:

```
auto (&ar)[2] = L"a"; // Array bound declared, element type deduced
```

This issue is related to issue 1892, which prohibited cases like

```
std::vector<auto(*)()> v;
```

The ultimate outcome of the two issues should be:

```
int f();
auto (*fp1)() = f;      // OK
auto (*fp2)()->int = f;  // OK
auto (*fp3)()->auto = f; // OK

template<typename T> struct C { };
C<auto(*)()> c1;         // Not OK
C<auto(*)()->int> c2;     // OK
C<auto(*)()->auto> c3;    // Not OK
```

Proposed resolution (January, 2023) [SUPERSEDED]:

1. Change in 9.2.9.7.1 [dcl.spec.auto.general] paragraph 1 as follows:

A *placeholder-type-specifier* designates a placeholder type that will be replaced later, **typically** by deduction from an initializer.

2. Change and split 9.2.9.7.1 [dcl.spec.auto.general] paragraph 3 as follows:

A placeholder type can appear ~~with a function declarator in the *decl-specifier-seq*, *type-specifier-seq*, *conversion-function-id*, or *trailing-return-type*, in any context where such~~ **for a function declarator is valid that includes a *trailing-return-type* (9.3.4.6 [dcl.fct]).** If the function declarator includes a *trailing-return-type* (9.3.4.6 [dcl.fct]), that *trailing-return-type* specifies the declared return type of the function.

~~Otherwise, the~~ **A placeholder type can appear in the *decl-specifier-seq* or *type-specifier-seq* in the declared return type of a** function declarator ~~shall declare that declares~~ **a function.** If the declared return type of the function contains a placeholder type, the return type of the function is deduced from non-discarded return statements, if any, in the body of the function (8.5.2 [stmt.if]).

3. Change in 9.2.9.7.1 [dcl.spec.auto.general] paragraph 4 as follows:

The type of a variable declared using a placeholder type is deduced from its initializer. This use is allowed in an initializing declaration (9.4 [dcl.init]) of a variable. The placeholder type shall appear as one of the *decl-specifiers* in the *decl-specifier-seq* **and or as one of the *type-specifiers* in a *trailing-return-type* that specifies the type that replaces such a *decl-specifier*;** the *decl-specifier-seq* shall be followed by one or more *declarators*, each of which shall be followed by a non-empty *initializer*. [Example:

```
...
auto f() -> int;           // OK, f returns int
auto (*fp)() -> auto = f; // OK
...

-- end example ]
```

4. Change in 9.3.4.6 [dcl.fct] paragraph 1 as follows:

In a declaration `T D` where `D` has the form

```
D1 ( parameter-declaration-clause ) cv-qualifier-seqopt
  ref-qualifieropt noexcept-specifieropt attribute-specifier-seqopt trailing-return-typeopt
```

and the type of the contained *declarator-id* in the declaration `T D1` is "*derived-declarator-type-list* `T`";:

- If the *trailing-return-type* is present, `T` shall be the single *type-specifier* `auto`, and the declared return type of the function type is the type specified by the *trailing-return-type*.
- Otherwise, the declared return type of the function type is `T`.

~~the~~ **The** type of the *declarator-id* in `D` is "*derived-declarator-type-list* noexcept_{opt} function of parameter-type-list cv-qualifier-seq_{opt} ref-qualifier_{opt} returning `T` **U**", where

- the parameter-type-list is derived from the *parameter-declaration-clause* as described below,
- **U is the declared return type,** and
- the optional *noexcept* is present if and only if the exception specification (14.5 [except.spec]) is non-throwing.

The optional *attribute-specifier-seq* appertains to the function type.

5. Remove 9.3.4.6 [dcl.fct] paragraph 2:

~~In a declaration `T D` where `D` has the form~~

```
D1 ( parameter-declaration-clause ) cv-qualifier-seqopt
  ref-qualifieropt noexcept-specifieropt attribute-specifier-seqopt trailing-return-type
```


~~and the type ... The optional attribute-specifier-seq appertains to the function type.~~

6. Change in 11.4.8.3 [class.conv.fct] paragraph 1 as follows:

A declaration whose *declarator-id* has an *unqualified-id* that is a *conversion-function-id* declares a conversion function; its declarator shall be a function declarator (9.3.4.6 [dcl.fct]) of the form

~~ptr-declarator~~ **noPtr-declarator** (~~parameter-declaration-clause~~) ~~cv-qualifier-seq~~
~~ref-qualifier-seq~~~~opt~~ ~~noexcept-specifier~~~~opt~~ ~~attribute-specifier-seq~~~~opt~~ **parameters-and-qualifiers**

where the ~~ptr-declarator~~ **noPtr-declarator** consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms: ...

7. Change in 11.4.8.3 [class.conv.fct] paragraph 2 as follows:

A conversion function shall have no non-object parameters and shall be a non-static member function of a class or class template X; **its declared return type is the conversion-type-id** and it specifies a conversion from X to the type specified by the *conversion-type-id* interpreted as a *type-id* (9.3.2 [dcl.name]). A *decl-specifier* in the *decl-specifier-seq* of a conversion function (if any) shall not be a *defining-type-specifier*.

8. Remove 11.4.8.3 [class.conv.fct] paragraph 3:

~~The type of the conversion function is “noexcept~~opt~~ function taking no parameter cv-qualifier-seq~~opt~~ ref-qualifier~~opt~~ returning conversion-type-id”.~~

CWG 2023-06

This does not address `void f2(auto (*)() -> auto);`

Proposed resolution (approved by CWG 2023-11-10):

1. Change in 9.2.9.7.1 [dcl.spec.auto.general] paragraph 1 as follows:

A *placeholder-type-specifier* designates a placeholder type that will be replaced later, **typically** by deduction from an initializer.

2. Change 9.2.9.7.1 [dcl.spec.auto.general] paragraph 2 as follows:

A The type of a parameter-declaration of a function declaration (9.3.4.6 [dcl.fct]), lambda-expression (7.5.5 [expr.prim.lambda]), or template-parameter (13.2 [temp.param]) can be declared using a placeholder-type-specifier of the form *type-constraint*_{opt} *auto* **can be used as a decl-specifier of the decl-specifier-seq of a parameter-declaration of a function declaration or lambda-expression and, if it is not the auto type-specifier introducing. The placeholder type shall appear as one of the decl-specifiers in the decl-specifier-seq or as one of the type-specifiers in a trailing-return-type ; that specifies the type that replaces such a *decl-specifier* (see below); the placeholder type is a generic parameter type placeholder of the function declaration ~~or, lambda-expression, or template-parameter, respectively.~~**

3. Change and split 9.2.9.7.1 [dcl.spec.auto.general] paragraph 3 as follows:

A placeholder type can appear ~~with a function declarator~~ in the *decl-specifier-seq*, *type-specifier-seq*, *conversion-function-id*, or *trailing-return-type*, in any context where such ~~for a function declarator~~ is valid **that includes a trailing-return-type (9.3.4.6 [dcl.fct]).** If the function declarator includes a *trailing-return-type* (9.3.4.6 [dcl.fct]), that *trailing-return-type* specifies the declared return type of the function.

~~Otherwise, the~~ **A placeholder type can appear in the decl-specifier-seq or type-specifier-seq in the declared return type of a** function declarator ~~shall declare that declares a function. If the declared return type of the function contains a placeholder type;~~ ; the return type of the function is deduced from non-discarded return statements, if any, in the body of the function (8.5.2 [stmt.if]).

4. Change in 9.2.9.7.1 [dcl.spec.auto.general] paragraph 4 as follows:

The type of a variable declared using a placeholder type is deduced from its initializer. This use is allowed in an initializing declaration (9.4 [dcl.init]) of a variable. The placeholder type shall appear as one of the *decl-specifiers* in the *decl-specifier-seq* ~~and or as one of the type-specifiers in a trailing-return-type that specifies the type that replaces such a decl-specifier;~~ the *decl-specifier-seq* shall be followed by one or more *declarators*, each of which shall be followed by a non-empty *initializer*. [Example:

```
...  
auto f() -> int;           // OK, f returns int  
auto (*fp)() -> auto = f; // OK  
...  
-- end example ]
```

5. Change and split 9.2.9.7.1 [dcl.spec.auto.general] paragraph 5 as follows:

A placeholder type can also be used in the *type-specifier-seq* ~~in of the new-type-id or in the type-id of a new-expression (7.6.2.8 [expr.new])~~ **and as a decl-specifier of the parameter-declaration's decl-specifier-seq in a template-parameter (13.2 [temp.param]). In such a type-id, the placeholder type shall appear as one of the type-specifiers in the type-specifier-seq or as one of the type-specifiers in a trailing-return-type that specifies the type that replaces such a type-specifier.**

The *auto type-specifier* can also be used as the *simple-type-specifier* in an explicit type conversion (functional notation) (7.6.1.4 [expr.type.conv]).

6. Change in 9.3.4.6 [dcl.fct] paragraph 1 as follows:

In a declaration `T D` where **T may be empty and** `D` has the form

D1 (*parameter-declaration-clause*) *cv-qualifier-seq*_{opt}
*ref-qualifier*_{opt} *noexcept-specifier*_{opt} *attribute-specifier-seq*_{opt} **trailing-return-type**_{opt}

a derived-declarator-type-list is determined as follows: ~~and~~

- If the *unqualified-id* of the *declarator-id* is a *conversion-function-id*, the *derived-declarator-type-list* is empty.
- **Otherwise, the derived-declarator-type-list is as appears in** the type **"derived-declarator-type-list T"** of the contained *declarator-id* in the declaration T D1 ~~is "derived-declarator-type-list T"~~

The declared return type U of the function type is determined as follows:

- If the *trailing-return-type* is present, T shall be the single *type-specifier* *auto*, and U is the type specified by the *trailing-return-type*.
- Otherwise, if the declaration declares a conversion function, see 11.4.8.3 [class.conv.fct].
- Otherwise, U is T.

~~the~~**The** type of the *declarator-id* in D is "*derived-declarator-type-list* *noexcept*_{opt} function of *parameter-type-list* *cv-qualifier-seq*_{opt} *ref-qualifier*_{opt} returning **T** **U**", where

- the *parameter-type-list* is derived from the *parameter-declaration-clause* as described below and
- the optional *noexcept* is present if and only if the exception specification (14.5 [except.spec]) is non-throwing.

The optional *attribute-specifier-seq* appertains to the function type.

7. Remove 9.3.4.6 [dcl.fct] paragraph 2:

~~In a declaration T D where D has the form~~

~~D1 (*parameter-declaration-clause*) *cv-qualifier-seq*_{opt}
*ref-qualifier*_{opt} *noexcept-specifier*_{opt} *attribute-specifier-seq*_{opt} *trailing-return-type*_{opt}~~

~~and the type ... The optional attribute-specifier-seq appertains to the function type.~~

8. Change in 11.4.8.3 [class.conv.fct] paragraph 1 as follows:

A declaration whose *declarator-id* has an *unqualified-id* that is a *conversion-function-id* declares a conversion function; its declarator shall be a function declarator (9.3.4.6 [dcl.fct]) of the form

~~*ptr-declarator* **noptr-declarator** (*parameter-declaration-clause*) *cv-qualifier-seq*_{opt}
*ref-qualifier-seq*_{opt} *noexcept-specifier*_{opt} *attribute-specifier-seq*_{opt} *parameters-and-qualifiers*~~

where the ~~*ptr-declarator*~~ **noptr-declarator** consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms: ...

9. Change in 11.4.8.3 [class.conv.fct] paragraph 2 as follows:

A conversion function shall have no non-object parameters and shall be a non-static member function of a class or class template X; **its declared return type is the conversion-type-id** and it specifies a conversion from X to the type specified by the *conversion-type-id* interpreted as a *type-id* (9.3.2 [dcl.name]). A *decl-specifier* in the *decl-specifier-seq* of a conversion function (if any) shall not be a *defining-type-specifier*.

10. Remove 11.4.8.3 [class.conv.fct] paragraph 3:

~~The type of the conversion function is "noexcept_{opt} function taking no parameter *cv-qualifier-seq*_{opt} *ref-qualifier*_{opt} returning *conversion-type-id*".~~

2533. Storage duration of implicitly created objects

Section: 6.7.5 [basic.stc] **Status:** ready **Submitter:** Andrey Erokhin **Date:** 2022-02-17

In subclause 6.7.2 [intro.object] paragraph 10, operations implicitly creating objects are defined:

Some operations are described as *implicitly creating objects* within a specified region of storage. For each operation that is specified as implicitly creating objects, that operation implicitly creates and starts the lifetime of zero or more objects of implicit-lifetime types (6.8.1 [basic.types.general]) in its specified region of storage if...

However, the standard does not specify the storage duration that such an implicitly-created object has; this new method of object creation is not mentioned in 6.7.5.1 [basic.stc.general] paragraph 2:

Static, thread, and automatic storage durations are associated with objects introduced by declarations (6.2 [basic.def]) and implicitly created by the implementation (6.7.7 [class.temporary]). The dynamic storage duration is associated with objects created by a *new-expression* (7.6.2.8 [expr.new]).

With the exception of `malloc`, the storage duration should probably be that of the object providing storage (if any), similar to the provision for subobjects in 6.7.5.6 [basic.stc.inherit]:

The storage duration of subobjects and reference members is that of their complete object (6.7.2 [intro.object]).

The storage duration of an object created by a non-allocating form of an allocation function (17.6.3.4 [new.delete.placement]) should be treated similarly.

Possible resolution [SUPERSEDED]:

1. Change in 6.7.2 [intro.object] paragraph 13 as follows:

Any implicit or explicit invocation of a function named `operator new` or `operator new[]` implicitly creates objects **with dynamic storage duration** in the returned region of storage and returns a pointer to a suitable created object.

2. Change in 6.7.5.1 [basic.stc.general] paragraph 2 as follows:

Static, thread, and automatic storage durations are associated with objects introduced by declarations (6.2 [basic.def]) and implicitly created by the implementation (6.7.7 [class.temporary]). The dynamic storage duration is associated with objects created ~~by a new-expression (7.6.2.8 [expr.new])~~ in storage returned by an allocation function (6.7.5.5.2 [basic.stc.dynamic.allocation]) **other than a non-allocating form (17.6.3.4 [new.delete.placement])** or by C library memory allocation (20.2.12 [c.malloc]).

3. Change in 6.7.5.5.2 [basic.stc.dynamic.allocation] paragraph 3 as follows:

For an allocation function ~~other than a reserved-placement allocation function~~ **other than a non-allocating form** (17.6.3.4 [new.delete.placement]), the pointer returned on a successful call shall represent the address of storage that is aligned as follows:

4. Change in 6.7.5.6 [basic.stc.inherit] paragraph 1 as follows:

The storage duration of ~~subobjects and~~ reference members is that of their complete object. **The storage duration of an object nested within another object x is the storage duration of x** (6.7.2 [intro.object]).

5. Change in 7.6.2.8 [expr.new] paragraph 9 as follows:

An object created by a new-expression that invokes an allocation function with a non-allocating form (see below) has the storage duration of the object that used to occupy the region of storage where the new object is created. Objects Any other object created by a new-expression ~~have~~ **has** dynamic storage duration (6.7.5.5 [basic.stc.dynamic]). [Note 5: The lifetime of such an object is not necessarily restricted to the scope in which it is created. —end note]

6. Change in 20.2.12 [c.malloc] paragraph 4 as follows:

These functions implicitly create objects (6.7.2 [intro.object]) **with dynamic storage duration** in the returned region of storage and return a pointer to a suitable created object. In the case of calloc and realloc, the objects are created before the storage is zeroed or copied, respectively.

Additional note (December, 2023)

The approach outlined above is incomplete and the wrong direction. The concept of storage duration determines when an object is created and destroyed; for dynamic storage duration, the object is created and destroyed by explicit program action.

Proposed resolution (approved by CWG 2024-01-19):

1. Change in 6.7.5.1 [basic.stc.general] paragraph 2 as follows:

Static, thread, and automatic storage durations are associated with objects introduced by declarations (6.2 [basic.def]) and ~~implicitly created by the implementation~~ **with temporary objects** (6.7.7 [class.temporary]). The dynamic storage duration is associated with objects created by a new-expression (7.6.2.8 [expr.new]) **or with implicitly created objects (6.7.2 [intro.object]).**

2546. Defaulted secondary comparison operators defined as deleted

Section: 11.10.4 [class.compare.secondary] Status: ready Submitter: Jim X Date: 2022-03-07

(See also editorial issues 5335 and 5336.)

Consider the example in 11.10.4 [class.compare.secondary] paragraph 3:

```
struct HasNoLessThan { };
struct C {
    friend HasNoLessThan operator<=>(const C&, const C&);
    bool operator<(const C&) const = default; // OK, function is deleted
};
```

While the comment may reflect the intent, it does not follow from the wording. 11.10.4 [class.compare.secondary] paragraph 2 specifies:

The operator function with parameters x and y is defined as deleted if

- overload resolution (12.2 [over.match]), as applied to x @ y, does not result in a usable candidate, or
- the candidate selected by overload resolution is not a rewritten candidate.

Otherwise, the operator function yields x @ y. The defaulted operator function is not considered as a candidate in the overload resolution for the @ operator.

Overload resolution applied to x < y results in a usable candidate operator<=> (12.2.1 [over.match.general]) and that candidate is a rewritten candidate (12.2.2.3 [over.match.oper] bullet 3.4), thus operator< in the above example is not deleted. However, its definition is ill-formed, because the rewrite (x <=> y) < 0 is ill-formed (12.2.2.3 [over.match.oper] paragraph 8).

There is implementation divergence.

Subclause 11.10.3 [class.spaceship] paragraph 1 seems to prefer an ill-formed program for similar synthesized situations:

[Note 1: A synthesized three-way comparison is ill-formed if overload resolution finds usable candidates that do not otherwise meet the requirements implied by the defined expression. —end note]

Proposed resolution (approved by CWG 2024-03-20):

Change in 11.10.4 [class.compare.secondary] paragraph 2 as follows:

The operator function with parameters x and y is defined as deleted if

- a first overload resolution (12.2 [over.match]), as applied to $x @ y$,
 - does not result in a usable candidate, or
 - the selected candidate selected by overload resolution is not a rewritten candidate; **or**
- a second overload resolution for the expression resulting from the interpretation of $x @ y$ using the selected rewritten candidate (12.2.2.3 [over.match.oper]) does not result in a usable candidate (for example, that expression might be $(x <=> y) @ \emptyset$), or
- $x @ y$ cannot be implicitly converted to `bool`.

In any of the two overload resolutions above, the defaulted operator function is not considered as a candidate for the `@` operator. Otherwise, the operator function yields $x @ y$. ~~The defaulted operator function is not considered as a candidate in the overload resolution for the `@` operator.~~

2547. Defaulted comparison operator function for non-classes

Section: 9.5.2 [dcl.fct.def.default] Status: ready Submitter: Jim X Date: 2022-03-07

(See editorial issue 5337.)

Subclause 9.5.2 [dcl.fct.def.default] paragraph 1 specifies:

A function definition whose *function-body* is of the form `= default` ; is called an *explicitly-defaulted definition*. A function that is explicitly defaulted shall

- be a special member function or a comparison operator function (12.4.3 [over.binary]), and
- not have default arguments.

There seem to be no further restrictions on which comparison operator functions are allowed to be defaulted. For example,

```
enum E { };
bool operator==(E, E) = default; // well-formed?
```

Subclause 11.10.1 [class.compare.default] paragraph 1 applies only to comparison operator functions "for some class":

A defaulted comparison operator function (12.4.3 [over.binary]) for some class C shall be a non-template function that is

- a non-static const non-volatile member of C having one parameter of type `const C&` and either no *ref-qualifier* or the *ref-qualifier* `&`, or
- a friend of C having either two parameters of type `const C&` or two parameters of type C .

Proposed resolution [SUPERSEDED]:

1. Change in 9.5.2 [dcl.fct.def.default] paragraph 1 as follows:

A function definition whose *function-body* is of the form `= default` ; is called an *explicitly-defaulted definition*. A function that is explicitly defaulted shall

- be a special member function (11.4.4 [special]) or a comparison operator function (12.4.3 [over.binary], 11.10.1 [class.compare.default]), and
- not have default arguments (9.3.4.7 [dcl.fct.default]).

2. Change in 11.10.1 [class.compare.default] paragraph 1 as follows:

A defaulted comparison operator function (12.4.3 [over.binary]) ~~for some class C~~ shall be a non-template function that is

- a non-static const non-volatile member of **some class** C having one parameter of type `const C&` and either no *ref-qualifier* or the *ref-qualifier* `&`, or
- a friend of **some class** C having either two parameters of type `const C&` or two parameters of type C .

Such a comparison operator function is termed a comparison operator function for class C . A comparison operator function for class C that is defaulted on its first declaration ...

CWG 2023-12-01

A defaulted comparison function for an incomplete class later declared a friend for that class should be made ill-formed.

Proposed resolution (approved by CWG 2023-12-15):

1. Change in 9.5.2 [dcl.fct.def.default] paragraph 1 as follows:

A function definition whose *function-body* is of the form `= default` ; is called an *explicitly-defaulted definition*. A function that is explicitly defaulted shall

- be a special member function (11.4.4 [special]) or a comparison operator function (12.4.3 [over.binary], 11.10.1 [class.compare.default]), and
- not have default arguments (9.3.4.7 [dcl.fct.default]).

2. Change in 11.10.1 [[class.compare.default](#)] paragraph 1 as follows:

- A defaulted comparison operator function (12.4.3 [[over.binary](#)]) ~~for some class C~~ shall be a non-template function that ~~is~~
- ~~is~~ a non-static member or friend of ~~some class C~~,
 - ~~is defined as defaulted in C or in a context where C is complete~~, and
 - either has two parameters of type const C& or two parameters of type C, where the implicit object parameter (if any) is considered to be the first parameter.

Such a comparison operator function is termed a defaulted comparison operator function for class C. Name lookups in the implicit definition (9.5.2 [[decl.fct.def.default](#)]) of a comparison operator function are performed from a context equivalent to its function-body. A definition of a comparison operator as defaulted that appears in a class shall be the first declaration of that function. **[Example:**

```
struct S;
bool operator==(S, S) = default; // error: S is not complete
struct S {
    friend bool operator==(S, const S&) = default; // error: parameters of different types
};
enum E { };
bool operator==(E, E) = default; // error: not a member or friend of a class
```

-- end example]

2560. Parameter type determination in a *requirement-parameter-list*

Section: 7.5.7.1 [[expr.prim.req.general](#)] **Status:** ready **Submitter:** Daveed Vandevoorde **Date:** 2020-01-21

Consider:

```
template<typename T>
    requires requires (T p[10]) { (decltype(p))nullptr; }
int v = 42;
auto r = v<int>; // well-formed?
```

This example is only well-formed if the type of the parameter p is adjusted to T*, but the provisions in 9.3.4.6 [[decl.fct](#)] paragraph 5 cover function parameters only.

One option is to specify application of the same adjustments as for function parameters. Another option is to specify rules that arguably are more useful in a *requires-expression*.

Proposed resolution (approved by CWG 2023-11-07):

Change in 7.5.7.1 [[expr.prim.req.general](#)] paragraph 3 as follows:

A *requires-expression* may introduce local parameters using a *parameter-declaration-clause* (9.3.4.6 [[decl.fct](#)]). A local parameter of a *requires-expression* shall not have a default argument. **The type of such a parameter is determined as specified for a function parameter in 9.3.4.6 [[decl.fct](#)].** These parameters have no linkage, storage, or lifetime; they are only used as notation for the purpose of defining requirements. The *parameter-declaration-clause* of a *requirement-parameter-list* shall not terminate with an ellipsis.

[Example 2:

```
template<typename T>
concept C = requires(T t, ...) { // error: terminates with an ellipsis
    t;
};
template<typename T>
concept C2 = requires(T p[2]) {
    (decltype(p))nullptr; // OK, p has type "pointer to T"
};
```

—end example]

CWG 2023-06-17

There are arguments in favor of both options. Forwarded to EWG with [paper issue 1582](#).

EWG 2023-11-07

Accept the proposed resolution and forward to CWG for inclusion in C++26.

2568. Access checking during synthesis of defaulted comparison operator

Section: 11.10.1 [[class.compare.default](#)] **Status:** ready **Submitter:** Nicolai Josuttis **Date:** 2022-04-11

Consider:

```
struct Base {
protected:
```

```

    bool operator==(const Base& other) const = default;
};

struct Child : Base {
    int i;
    bool operator==(const Child& other) const = default;
};

```

Per 11.10.1 [class.compare.default] paragraph 6,

Let x_i be an lvalue denoting the i -th element in the expanded list of subobjects for an object x (of length n), where x_i is formed by a sequence of derived-to-base conversions (12.2.4.2 [over.best.ics]), class member access expressions (7.6.1.5 [expr.ref]), and array subscript expressions (7.6.1.2 [expr.sub]) applied to x .

The derived-to-base conversion for this loses the context of access to the protected `Base::operator==`, violating 11.8.5 [class.protected] paragraph 1. The example is rejected by implementations, but ought to work.

For this related example, there is implementation divergence:

```

struct B {
protected:
    constexpr operator int() const { return 0; }
};
struct D : B {
    constexpr bool operator==(const D&) const = default;
};
template<typename T> constexpr auto comparable(T t) -> decltype(t == t) { return t == t; }
constexpr bool comparable(...) { return false; }
static_assert(comparable(D{}));

```

Is `D::operator==` deleted, because its defaulted definition violates the protected access rules? Is `D::operator==` not deleted, but synthesis fails on use because of the protected access rules? Is the synthesis not in the immediate context, making the expression `comparable(D{})` ill-formed?

CWG 2023-06-17

There is no implementation divergence; the first example is intended to be well-formed.

Proposed resolution (approved by CWG 2023-12-01):

Change in 11.10.1 [class.compare.default] paragraph 1 as follows:

... Name lookups **and access checks** in the implicit definition (9.5.2 [dcl.fct.def.default]) of a comparison operator function are performed from a context equivalent to its *function-body*. A definition of a comparison operator as defaulted that appears in a class shall be the first declaration of that function.

2634. Avoid circularity in specification of scope for friend class declarations

Section: 9.2.9.5 [dcl.type.elab] Status: ready Submitter: Jim X Date: 2022-07-04

Consider:

```

auto f(struct X* ptr) {
    struct D {
        private:
            int d;
            friend class X;    // #1
    };
    return D{};
}
X* b = 0;
struct X {
    void show() {
        auto t = f(0);
        t.d = 10;            // #2 error: ::X is not a friend of f::D
    }
};

```

The target scope for #2 is `f`'s block scope, making `::X` not a friend of `f::D`. Thus the access at #2 is ill-formed. Clang disagrees.

Subclause 9.2.9.5 [dcl.type.elab] paragraph 3 specifies:

... If E contains an identifier but no *nested-name-specifier* and (unqualified) lookup for the identifier finds nothing, E shall not be introduced by the `enum` keyword and declares the identifier as a *class-name*. The target scope of E is the nearest enclosing namespace or block scope.

If an *elaborated-type-specifier* appears with the friend specifier as an entire *member-declaration*, the *member-declaration* shall have one of the following forms:

```

friend class-key nested-name-specifieropt identifier ;
...

```

Any unqualified lookup for the *identifier* (in the first case) does not consider scopes that contain the target scope; no name is bound.

This specification is circular in that the target scope that limits unqualified lookup is defined only if the *identifier* is actually declared, but the *identifier* is declared only if lookup finds nothing.

Proposed resolution (approved by CWG 2023-11-11):

Change in 9.2.9.5 [dcl.type.elab] paragraph 4 as follows:

... Any unqualified lookup for the *identifier* (in the first case) does not consider scopes that contain the ~~target~~ nearest enclosing namespace or block scope; no name is bound. [Note: ...]

2637. Injected-class-name as a *simple-template-id*

Section: 11.1 [class.pre] **Status:** ready **Submitter:** Shafik Yaghmour **Date:** 2022-10-26

Issue 2237 sought to disallow *simple-template-ids* as constructor names, by referring to the injected-class-name. However, 11.1 [class.pre] paragraph 2 specifies:

The *class-name* is also bound in the scope of the class (template) itself; this is known as the *injected-class-name*.

The grammar non-terminal *class-name* includes the option of a *simple-template-id* (for declaring a partial specialization).

Proposed resolution (approved by CWG 2023-11-11):

Change in 11.1 [class.pre] paragraph 2 as follows:

The **component name of the** *class-name* is also bound in the scope of the class (template) itself; this is known as the *injected-class-name*. ...

2638. Improve the example for initializing by initializer list

Section: 9.4.5 [dcl.init.list] **Status:** ready **Submitter:** Shafik Yaghmour **Date:** 2022-10-26

Issue 2137 amended the rules for initialization by initializer list, but neglected to add an example.

Proposed resolution (approved by CWG 2023-11-11):

Change the example in 9.4.5 [dcl.init.list] bullet 3.7 as follows:

```
struct S {
    S(std::initializer_list<double>); // #1
    S(std::initializer_list<int>);    // #2
    S(std::initializer_list<S>);      // #3
    S();                             // #3#4

    // ...
};
S s1 = { 1.0, 2.0, 3.0 }; // invoke #1
S s2 = { 1, 2, 3 };      // invoke #2
S s3{s2};                // invoke #3 (not the copy constructor)
S s3s4 = { };            // invoke #3#4
```

2657. Cv-qualification adjustment when binding reference to temporary

Section: 9.4.4 [dcl.init.ref] **Status:** ready **Submitter:** Brian Bi **Date:** 2022-11-10

Core issue 2481 was resolved by clarifying that the temporary object in p5.4.2 is cv-qualified if the reference being initialized is cv-qualified. However, this is not the right bullet point for the example given,

```
constexpr const int &r = 42;
```

Such an initialization would actually use bullet 5.3.1 instead. (5.4.2 would be used if the initializer were, for example, 3.14.) We therefore need to make a similar clarification in bullet 5.3, and ideally using the same language.

Proposed resolution (approved by CWG 2024-03-20):

1. Change in 9.4.4 [dcl.init.ref] bullet 5.3 as follows:

- Otherwise, if the initializer expression
 - is an rvalue (but not a bit-field) or function lvalue and “cv1 T1” is reference-compatible with “cv2 T2”, or

- has a class type (i.e., T2 is a class type), where T1 is not reference-related to T2, and can be converted to an rvalue or function lvalue of type “cv3 T3”, where “cv1 T1” is reference-compatible with “cv3 T3” (see 12.2.2.7 [\[over.match.ref\]](#)), then the initializer expression in the first case and the converted expression in the second case is called the converted initializer. If the converted initializer is a prvalue, **let its type be denoted by T4; the temporary materialization conversion (7.3.5 [\[conv.rval\]](#)) is applied, considering the type of the prvalue to be is adjusted to type “cv1 T4” (7.3.6 [\[conv.qual\]](#)) and the temporary materialization conversion (7.3.5 [\[conv.rval\]](#)) is applied**. In any case, the reference binds to the resulting glvalue (or to an appropriate base class subobject).

2. Append to the example in 9.4.4 [\[dcl.init.ref\]](#) bullet 5.3 as follows:

```
B&& rrb = x; // binds directly to the result of operator B

constexpr int f() {
    const int &x = 42;
    const_cast<int &>(x) = 1; // undefined behavior
    return x;
}
constexpr int z = f(); // error: not a constant expression
```

3. Change the example in 9.4.4 [\[dcl.init.ref\]](#) bullet 5.4 as follows:

```
const double& rcd2 = 2; // rcd2 refers to temporary with type const double and value 2.0
```

2661. Missing disambiguation rule for *pure-specifier* vs. *brace-or-equal-initializer*

Section: 11.4.1 [\[class.mem.general\]](#) **Status:** ready **Submitter:** Richard Smith **Date:** 2022-10-07

Subclause 11.4.1 [\[class.mem.general\]](#) has this grammar:

```
member-declarator:
    declarator virt-specifier-seq[opt] pure-specifier[opt]
    declarator brace-or-equal-initializer[opt]

pure-specifier:
    = 0
```

The primary issue is that `foo = 0` matches both *member-declarator* productions. Secondly, a *declarator* by itself is also ambiguous.

Code such as `virtual FunctionType f = 0;` can be valid, so disambiguation on the syntactic form of the *declarator* is not possible.

Proposed resolution (approved by CWG 2024-02-16):

Change and add before 11.4.1 [\[class.mem.general\]](#) paragraph 1 as follows:

```
member-declarator:
    declarator virt-specifier-seqopt pure-specifieropt
    declarator brace-or-equal-initializeropt
```

In the absence of a *virt-specifier-seq*, the token sequence `= 0` is treated as a *pure-specifier* if the type of the *declarator-id* (9.3.4.1 [\[dcl.meaning.general\]](#)) is a function type, and is otherwise treated as a *brace-or-equal-initializer*. [*Note:* If the member declaration acquires a function type through template instantiation, the program is ill-formed; see 13.9.1 [\[temp.spec.general\]](#). --end note]

2668. `co_await` in a *lambda-expression*

Section: 7.6.2.4 [\[expr.await\]](#) **Status:** ready **Submitter:** Jim X **Date:** 2022-12-12

Subclause 7.6.2.4 [\[expr.await\]](#) paragraph 2 disallows an *await-expression* to appear in the body of a *lambda-expression*:

An *await-expression* shall appear only in a potentially-evaluated expression within the *compound-statement* of a *function-body* outside of a handler (14.1 [\[except.pre\]](#)). ...

This is probably unintended.

Proposed resolution (approved by CWG 2023-11-11):

Change in 7.6.2.4 [\[expr.await\]](#) paragraph 2 as follows:

An *await-expression* shall appear only **in** **as** a potentially-evaluated expression within the *compound-statement* of a *function-body* **or lambda-expression, in either case** outside of a handler (14.1 [\[except.pre\]](#)). ...

2689. Are cv-qualified `std::nullptr_t` fundamental types?

It is unclear whether `cv std::nullptr_t` is a fundamental type, given that it is declared in a library header and cv-qualifications are not mentioned in 6.8.2 [basic.fundamental] paragraph 15.

Proposed resolution (approved by CWG 2023-12-01):

Change in 6.8.2 [basic.fundamental] paragraph 15 as follows:

The types denoted by `cv std::nullptr_t` are distinct types. A value of type `std::nullptr_t` is a null pointer constant (7.3.12 [conv.ptr]). Such values participate in the pointer and the pointer-to-member conversions (7.3.12 [conv.ptr], 7.3.13 [conv.mem]). `sizeof(std::nullptr_t)` shall be equal to `sizeof(void*)`.

The types described in this subclause are called *fundamental types*. [Note 11: Even if the implementation defines two or more fundamental types to have the same value representation, they are nevertheless different types. —end note]

2700. #error disallows existing implementation practice

The resolution for issue 2518 disallows existing implementation practice, as detailed below:

- The requirement to issue a diagnostic message for each appearance of `#warning` conflicts with existing implementations having a diagnostic limit. Furthermore, an implementation with an on-the-fly incremental preprocessor is no longer allowed to abort the compilation when an error deemed fatal happens (e.g. a `#include` file that does not exist).
- The wording does not allow for implementation-specific mechanisms for creating a preprocessing translation unit, such as vendor-specific forms of `#if`-like directives and `#pragma once`.

Suggested resolution [SUPERSEDED]:

1. Change in 4.1.1 [intro.compliance.general] paragraph 2 as follows:

Furthermore, a conforming implementation

- shall not accept a preprocessing translation unit containing a `#error` preprocessing directive (15.8 [cpp.error]); **and**
- ~~shall issue at least one diagnostic message for each `#warning` or `#error` preprocessing directive not following a `#error` preprocessing directive in a preprocessing translation unit, and~~
- shall not accept a translation unit with a `static_assert`-declaration that fails (9.1 [dcl.pre]).

2. Change in 5.1 [lex.separate] paragraph 1 as follows:

The text of the program is kept in units called source files in this document. A source file together with all the headers (16.4.2.3 [headers]) and source files included (15.3 [cpp.include]) via the preprocessing directive `#include`, less any source lines skipped by any of the conditional inclusion (15.2 [cpp.cond]) preprocessing directives **or by the implementation-defined behavior of any conditionally-supported-directives (15.1 [cpp.pre])**, is called a *preprocessing translation unit*.

CWG 2023-03-03

Permit that `#warning` can be ignored if another diagnostic is produced.

Proposed resolution (approved by CWG 2024-01-19):

1. Change in 4.1.1 [intro.compliance.general] bullet 2.3 as follows:

- ...
- Otherwise, if a program contains
 - a violation of any diagnosable rule ~~or,~~
 - **a preprocessing translation unit with a `#warning` preprocessing directive (15.8 [cpp.error]), or**
 - an occurrence of a construct described in this document as “conditionally-supported” when the implementation does not support that construct,a conforming implementation shall issue at least one diagnostic message.

2. Change in 4.1.1 [intro.compliance.general] paragraph 2 as follows:

Furthermore, a conforming implementation shall not accept

- a preprocessing translation unit containing a `#error` preprocessing directive (15.8 [cpp.error]); **or**
- ~~shall issue at least one diagnostic message for each `#warning` or `#error` preprocessing directive not following a `#error` preprocessing directive in a preprocessing translation unit, and~~
- ~~shall not accept~~ a translation unit with a `static_assert`-declaration that fails (9.1 [dcl.pre]).

3. Change in 5.1 [lex.separate] paragraph 1 as follows:

The text of the program is kept in units called source files in this document. A source file together with all the headers (16.4.2.3 [headers]) and source files included (15.3 [cpp.include]) via the preprocessing directive `#include`, less any source lines skipped by any of the conditional inclusion (15.2 [cpp.cond]) preprocessing directives, **as modified by the implementation-defined behavior of any conditionally-supported-directives (15.1 [cpp.pre]) and pragmas (15.9 [cpp.pragma]), if any**, is called a *preprocessing translation unit*.

4. Change in 15.8 [cpp.error] as follows:

A preprocessing directive of ~~either of the following forms~~ **the form**

`# error pp-tokensopt new-line`

renders the program ill-formed. A preprocessing directive of the form

`# warning pp-tokensopt new-line`

~~causes~~ **requires** the implementation to produce ~~a~~ **at least one** diagnostic message **for the preprocessing translation unit (4.1.1 [intro.compliance.general]) that**.

Recommended practice: Any diagnostic message caused by either of these directives should include the specified sequence of preprocessing tokens; ~~the #error directive renders the program ill-formed.~~

2707. Deduction guides cannot have a trailing *requires-clause*

Section: 13.7.2.3 [temp.deduct.guide] **Status:** ready **Submitter:** Richard Smith **Date:** 2020-02-26

The grammar for *deduction-guide* does not, but should, allow a trailing *requires-clause*:

```
deduction-guide:
  explicit-specifieropt template-name ( parameter-declaration-clause ) -> simple-template-id ;
```

Proposed resolution (approved by CWG 2023-11-11):

Change the grammar in 13.7.2.3 [temp.deduct.guide] paragraph 1 as follows:

```
deduction-guide:
  explicit-specifieropt template-name ( parameter-declaration-clause ) requires-clauseopt -> simple-template-id ;
```

2714. Implicit deduction guides omit properties from the parameter-declaration-clause of a constructor

Section: 12.2.2.9 [over.match.class.deduct] **Status:** ready **Submitter:** Richard Smith **Date:** 2017-02-13

Subclause 12.2.2.9 [over.match.class.deduct] bullet 1.1.2 specifies:

- The types of the function parameters are those of the constructor.

However, this does not consider default arguments or variadic constructors.

Proposed resolution (approved by CWG 2023-04-28):

(This also resolves issue 2628.)

Change in 12.2.2.9 [over.match.class.deduct] paragraph 1 as follows:

- If C is defined, for each constructor of C, a function template with the following properties:
 - The template parameters are the template parameters of C followed by the template parameters (including default template arguments) of the constructor, if any.
 - **The associated constraints (13.5.3 [temp.constr.decl]) are the conjunction of the associated constraints of C and the associated constraints of the constructor, if any. [Note: A constraint-expression in the template-head of C is checked for satisfaction before any constraints from the template-head or trailing-requires-clause of the constructor. -- end note]**
 - The ~~types of the function parameters are those~~ **parameter-declaration-clause is that** of the constructor.
 - The return type is the class template specialization designated by C and template arguments corresponding to the template parameters of C.
- If C is not defined or does not declare any constructors, an additional function template derived as above from a hypothetical constructor C().
- An additional function template derived as above from a hypothetical constructor C(C), called the copy deduction candidate.
- For each *deduction-guide*, a function or function template with the following properties:
 - The ~~template parameters~~ **template-head**, if any, and ~~function parameters~~ **parameter-declaration-clause** are those of the *deduction-guide*.
 - The return type is the *simple-template-id* of the *deduction-guide*.

2745. Dependent odr-use in generic lambdas

Section: 6.3 [basic.def.odr] **Status:** ready **Submitter:** Shafik Yaghmour **Date:** 2022-12-13

Default template arguments of generic lambdas can refer to local variables. It is unclear whether the potential odr-use is checked when parsing the template definition or when instantiating the template.

There is wide implementation divergence.

Proposed resolution (approved by CWG 2024-03-20):

Insert a new paragraph before 6.3 [\[basic.def.odr\]](#) paragraph 11:

[Example:

```
void g() {
    constexpr int x = 1;
    auto lambda = [] <typename T, int = ((T)x, 0)> {}; // OK
    lambda.operator()<int, 1>(); // OK, does not consider x at all
    lambda.operator()<int>(); // OK, does not odr-use x
    lambda.operator()<const int&>(); // error: odr-uses x from a context where x is not odr-usable
}

void h() {
    constexpr int x = 1;
    auto lambda = [] <typename T> { (T)x; }; // OK
    lambda.operator()<int>(); // OK, does not odr-use x
    lambda.operator()<void>(); // OK, does not odr-use x
    lambda.operator()<const int&>(); // error: odr-uses x from a context where x is not odr-usable
}
```

-- end example]

Every program shall contain at least one definition of every function or variable ...

2746. Checking of default template arguments

Section: 13.8.1 [\[temp.res.general\]](#) **Status:** ready **Submitter:** Shafik Yaghmour **Date:** 2022-12-13

Consider:

```
static int x = 1;
template<auto y = x> void f() {}
```

Is the definition of `f` well-formed? Since `x` is not a constant expression, any use of the default template argument is ill-formed, but for example `f<5>()` does not actually use it.

Are implementations allowed or required to reject this situation, even if the template is never instantiated? If the default template argument is dependent, checking may need to be deferred to instantiations in any case.

Proposed resolution (approved by CWG 2024-03-01):

Change in 13.8.1 [\[temp.res.general\]](#) paragraph 6 as follows:

The validity of a template may be checked prior to any instantiation. *[Note : ... —end note]*

The program is ill-formed, no diagnostic required, if:

- no valid specialization, ignoring *static_assert-declarations* that fail, can be generated for a template or a substatement of a *constexpr* if statement (8.5.2 [\[stmt.if\]](#)) within a template and the template is not instantiated, or
- no valid specialization, ignoring *static_assert-declarations* that fail, can be generated for a default *template-argument* and the default *template-argument* is not used in any instantiation, or
- any *constraint-expression* in the program, introduced or otherwise, has (in its normal form) an atomic constraint *A* where no satisfaction check of *A* could be well-formed and no satisfaction check of *A* is performed, or
- every valid specialization of a variadic template requires an empty template parameter pack, or
- a hypothetical instantiation of a template immediately following its definition would be ill-formed due to a construct that does not depend on a template parameter, or
- the interpretation of such a construct in the hypothetical instantiation is different from the interpretation of the corresponding construct in any actual instantiation of the template.

2748. Accessing static data members via null pointer

Section: 7.6.1.5 [\[expr.ref\]](#) **Status:** ready **Submitter:** Tomasz Kamiński **Date:** 2023-06-13

Consider:

```
struct C { static int foo; };
C* c = nullptr;
```

The behavior of `(*c).foo` is clearly undefined per 7.6.1.5 [\[expr.ref\]](#) paragraph 1:

The postfix expression before the dot or arrow is evaluated ...

However, the treatment of `c->foo` is less clear, because the transformation to the form `(*(E1)).E2` occurs later.

Proposed resolution (approved by CWG 2023-12-15):

Move a part of 7.6.1.5 [expr.ref] paragraph 1 to before paragraph 3 and edit as follows:

A postfix expression followed by a dot `.` or an arrow `->`, optionally followed by the keyword template, and then followed by an *id-expression*, is a postfix expression. ~~The postfix expression before the dot or arrow is evaluated; [Footnote: ...] the result of that evaluation, together with the *id-expression*, determines the result of the entire postfix expression; [Note 1: If the keyword template is used, the following unqualified name is considered to refer to a template (13.3 [temp.names]). If a *simple-template-id* results and is followed by a `::`, the *id-expression* is a *qualified-id*. —end note]~~

For the first option (dot) the first expression shall be a glvalue. For the second option (arrow) the first expression shall be a prvalue having pointer type. The expression `E1->E2` is converted to the equivalent form `(*(E1)).E2`; the remainder of 7.6.1.5 [expr.ref] will address only the first option (dot). [Footnote: ...]

The postfix expression before the dot is evaluated; [Footnote: ...] the result of that evaluation, together with the *id-expression*, determines the result of the entire postfix expression.

2771. Transformation for *unqualified-ids* in address operator

Section: 11.4.3 [class.mfct.non.static] Status: ready Submitter: Jim X Date: 2023-07-16

Consider:

```
struct A{
  int a;
  void show(){
    int* r = &a; // #1
  }
};
```

According to 11.4.3 [class.mfct.non.static] paragraph 2, the transformation to class member access does not happen for the *id-expression* `a`, because it is the unparenthesized operand of `&`:

When an *id-expression* (7.5.4 [expr.prim.id]) that is neither part of a class member access syntax (7.6.1.5 [expr.ref]) nor the un-parenthesized operand of the unary `&` operator (7.6.2.2 [expr.unary.op]) is used where the current class is `X` (7.5.2 [expr.prim.this]), if name lookup (6.5 [basic.lookup]) resolves the name in the *id-expression* to a non-static non-type member of some class `C`, and if either the *id-expression* is potentially evaluated or `C` is `X` or a base class of `X`, the *id-expression* is transformed into a class member access expression (7.6.1.5 [expr.ref]) using `(*this)` as the *postfix-expression* to the left of the `.` operator. [Note 1: If `C` is not `X` or a base class of `X`, the class member access expression is ill-formed. —end note] This transformation does not apply in the template definition context (13.8.3.2 [temp.dep.type]).

Proposed resolution (approved by CWG 2024-03-20):

This resolution moves the transformation to 7.5.4.1 [expr.prim.id.general], where the similar transformation for anonymous unions is already described.

1. Change in 6.3 [basic.def.odr] paragraph 7 as follows:

~~`*this` is odr-used if this appears as a potentially-evaluated expression (including as the result of the **any** implicit transformation in the body of a non-static member function to a class member access expression (11.4.3 [class.mfct.non.static] 7.5.4.1 [expr.prim.id.general])).~~

2. Add a new paragraph before 7.5.4.1 [expr.prim.id.general] paragraph 2:

If an *id-expression* `E` denotes a non-static non-type member of some class `C` at a point where the current class (7.5.2 [expr.prim.this]) is `X` and

- `E` is potentially evaluated or `C` is `X` or a base class of `X`, and
- `E` is not the *id-expression* of a class member access expression (7.6.1.5 [expr.ref]), and
- if `E` is a *qualified-id*, `E` is not the un-parenthesized operand of the unary `&` operator (7.6.2.2 [expr.unary.op]),

the *id-expression* is transformed into a class member access expression using `(*this)` as the object expression. [Note 1: If `C` is not `X` or a base class of `X`, the class member access expression is ill-formed. Also, if the *id-expression* occurs within a static or explicit object member function, the class member access is ill-formed. —end note] This transformation does not apply in the template definition context (13.8.3.2 [temp.dep.type]).

If an *id-expression* `E` denotes a member `M` of an anonymous union (11.5.2 [class.union.anon]) `U`:

- If `U` is a non-static data member, `E` refers to `M` as a member of the lookup context of the terminal name of `E` (after any **implicit** transformation to a class member access expression (11.4.3 [class.mfct.non.static])).
- ...

3. Change in 7.5.4.1 [expr.prim.id.general] paragraph 3 as follows:

An *id-expression* that denotes a non-static data member or implicit object member function of a class can only be used:

- as part of a class member access (7.6.1.5 [expr.ref] **after any implicit transformation (see above)**) in which the object expression refers to the member's class [Footnote: This also applies when the object expression is an implicit `(*this)` (11.4.3 [class.mfct.non.static])]. or a class derived from that class, or

o ...

4. Change in 7.5.4.2 [expr.prim.id.unqual] paragraph 1 as follows:

... [Note: ... Within the definition of a non-static member function, an identifier that names a non-static member is transformed to a class member access expression (11.4.3 [class.mfct.non.static]); —end note]

5. Remove 11.4.3 [class.mfct.non.static] paragraph 2, including the example:

When an *id-expression* (7.5.4 [expr.prim.id]) that is neither part of a class member access syntax (7.6.1.5 [expr.ref]) nor the un-parenthesized operand of the unary & operator (7.6.2.2 [expr.unary.op]) is used where the current class is X (7.5.2 [expr.prim.this]), if name lookup (6.5 [basic.lookup]) resolves the name in the *id-expression* to a non-static non-type member of some class C, and if either the *id-expression* is potentially evaluated or C is X or a base class of X, the *id-expression* is transformed into a class member access expression (7.6.1.5 [expr.ref]) using (*this) as the *postfix-expression* to the left of the . operator. [Note 1: If C is not X or a base class of X, the class member access expression is ill-formed. —end note] This transformation does not apply in the template definition context (13.8.3.2 [temp.dep.type]). [Example: ...]

6. Change in 13.8.3.2 [temp.dep.type] paragraph 6 as follows:

[Example: ...

```
template int C<B>::g();    // OK, transformation to class member access syntax
                          // does not occur in the template definition context; see 11.4.3 [class.mfct.non.static] 7.5.4.1 [expr.prim.id.general]
```

-- end example]

2775. Unclear argument type for copy of exception object

Section: 14.2 [except.throw] Status: ready Submitter: Jiang An Date: 2023-05-31

Subclause 14.2 [except.throw] paragraph 5 specifies:

When the thrown object is a class object, the constructor selected for the copy-initialization as well as the constructor selected for a copy-initialization considering the thrown object as an lvalue shall be non-deleted and accessible, even if the copy/move operation is elided (11.9.6 [class.copy.elision]). The destructor is potentially invoked (11.4.7 [class.dtor]).

This provision is for capturing a copy constructor for implementations not using reference-counted std::exception_ptrs, but that ought to be described separately from the "thrown object", which can be interpreted as the operand of the *throw-expression*.

Proposed resolution (approved by CWG 2023-09-15) [SUPERSEDED]:

Change in 14.2 [except.throw] paragraph 5 as follows:

When the thrown object is a class object, the constructor selected for the copy-initialization as well as the constructor selected for a copy-initialization considering the thrown object as an lvalue shall be non-deleted and accessible, even if the copy/move operation is elided (11.9.6 [class.copy.elision]). **Let T denote the type of the exception object. Copy-initialization of an object of type T from an lvalue of type const T in a context unrelated to any class shall be well-formed. If T is a class type, the the destructor of T is potentially invoked (11.4.7 [class.dtor]).**

CWG 2023-11-09

The drafting should also consider odr-use of the constructor potentially invoked for the copy-initialization.

Proposed resolution (approved by CWG 2023-11-10) [SUPERSEDED]:

1. Change in 6.3 [basic.def.odr] paragraph 9 as follows:

An assignment operator function in a class is odr-used by an implicitly-defined copy assignment or move assignment function for another class as specified in 11.4.6 [class.copy.assign]. A constructor for a class is odr-used as specified in 9.4 [dcl.init] **and when selected for the potential copy-initialization as specified in 14.2 [except.throw].** A destructor for a class is odr-used if it is potentially invoked (11.4.7 [class.dtor]).

2. Change in 14.2 [except.throw] paragraph 5 as follows:

When the thrown object is a class object, the constructor selected for the copy-initialization as well as the constructor selected for a copy-initialization considering the thrown object as an lvalue shall be non-deleted and accessible, even if the copy/move operation is elided (11.9.6 [class.copy.elision]). **Let T denote the type of the exception object. Copy-initialization of an object of type T from an lvalue of type const T in a context unrelated to T shall be well-formed. If T is a class type, the the destructor of T is potentially invoked (11.4.7 [class.dtor]).**

Proposed resolution (approved by CWG 2024-03-20):

1. Change in 14.2 [except.throw] paragraph 5 as follows:

When the thrown object is a class object, the constructor selected for the copy-initialization as well as the constructor selected for a copy-initialization considering the thrown object as an lvalue shall be non-deleted and accessible, even if the copy/move operation is elided (11.9.6 [class.copy.elision]). **Let T denote the type of the exception object. Copy-initialization of an object of type T from an lvalue of type const T in a context unrelated to T shall be well-formed. If T is a class type, the selected constructor is odr-used (6.3 [basic.def.odr]) and the the destructor of T is potentially invoked (11.4.7 [class.dtor]).**

2777. Type of *id-expression* denoting a template parameter object

Section: 13.2 [\[temp.param\]](#) **Status:** ready **Submitter:** Jim X **Date:** 2023-07-26

The type of a template parameter object is specified to be `const T` in 13.2 [\[temp.param\]](#) paragraph 8:

An *id-expression* naming a non-type *template-parameter* of class type `T` denotes a static storage duration object of type `const T`, known as a template parameter object, whose value is that of the corresponding template argument after it has been converted to the type of the *template-parameter*. ...

However, it is unclear what the type of an *id-expression* is that refers to such an object. There is implementation divergence in the treatment of the following example:

```
struct A {};  
  
template<auto a, auto x> // also consider A a and const auto x  
int f() {  
    decltype(a) b;      // also consider decltype((a))  
    A& rb = b;  
    decltype(x) y;  
    int& ry = y;  
}  
  
int x = f<A{}>, 42>();
```

Note that non-type template parameters are handled specially for `decltype`, as specified in 9.2.9.6 [\[decl.type.decltype\]](#) paragraph 1:

For an expression `E`, the type denoted by `decltype(E)` is defined as follows:

- ...
- otherwise, if `E` is an unparenthesized *id-expression* naming a non-type *template-parameter* (13.2 [\[temp.param\]](#)), `decltype(E)` is the type of the *template-parameter* after performing any necessary type deduction (9.2.9.7 [\[decl.spec.auto\]](#), 9.2.9.8 [\[decl.type.class.deduct\]](#));
- ...

Proposed resolution (approved by CWG 2024-03-01):

Change in 7.5.4.2 [\[expr.prim.id.unqual\]](#) paragraph 3 as follows:

... ~~[Note 4: If the entity is a template parameter object for a template parameter of type `T` (13.2 [\[temp.param\]](#)), the type of the expression is `const T`.~~
~~end note]~~ **In all other cases, the type of the expression is the type of the entity.**

2803. Overload resolution for reference binding of similar types

Section: 12.2.4.2.5 [\[over.ics.ref\]](#) **Status:** ready **Submitter:** Brian Bi **Date:** 2023-06-14

Consider:

```
int foo(int*& r);          // #1  
int foo(const int* const& r); // #2  
  
int *p;  
int x = foo(p);
```

Both #1 and #2 perform direct reference binding; no qualification conversions are involved. Despite the lack of a rule, implementations prefer #1 over #2.

Proposed resolution (approved by CWG 2023-11-10):

1. Change in 12.2.4.2.5 [\[over.ics.ref\]](#) paragraph 1 as follows:

When a parameter of reference type **"reference to cv T"** binds directly (9.4.4 [\[decl.init.ref\]](#)) to an argument expression, ~~the implicit conversion sequence is the identity conversion, unless:~~

- **If** the argument expression has a type that is a derived class of the parameter type, ~~in which case~~ the implicit conversion sequence is a derived-to-base conversion (12.2.4.2 [\[over.best.ics\]](#)).
- Otherwise, if `T` is a function type, or if the type of the argument is possibly cv-qualified `T`, or if `T` is an array type of unknown bound with element type `U` and the argument has an array type of known bound whose element type is possibly cv-qualified `U`, the implicit conversion sequence is the identity conversion. [*Note:* When `T` is a function type, the type of the argument may differ only by the presence of `noexcept`. -- end note]
- Otherwise, the implicit conversion sequence is a qualification conversion.

[*Example 1:* ... —end example]

If the parameter binds directly to the result of applying a conversion function to the argument expression, the implicit conversion sequence is a user-defined conversion sequence (12.2.4.2.3 [\[over.ics.user\]](#)) whose second standard conversion sequence is ~~either an identity conversion or, if the conversion function returns an entity of a type that is a derived class of the parameter type, a derived-to-base conversion~~ **determined by the above rules.**

2. Change in 12.2.4.3 [over.ics.rank] bullet 3.2.5 as follows:

- S1 and S2 differ only in their qualification conversion (7.3.6 [conv.qual]) and yield similar types T1 and T2, respectively (where a standard conversion sequence that is a reference binding is considered to yield the cv-unqualified referenced type), where T1 can be converted to T2 by a qualification conversion and T2 are not the same type, and const T2 is reference-compatible with T1 (9.4.4 [dcl.init.ref]). [Example 5:

```
int f(const volatile int *);
int f(const int *);
int i;
int j = f(&i); // calls f(const int*)
int g(const int*);
int g(const volatile int* const&);
int* p;
int k = g(p); // calls g(const int*)
```

-- end example] or, if not that,

3. Change in 12.2.4.3 [over.ics.rank] bullet 3.2.6 as follows:

- S1 and S2 include reference bindings bind "reference to T1" and "reference to T2", respectively (9.4.4 [dcl.init.ref]), and the types to which the references refer are the same type except for top-level cv-qualifiers, and the type to which the reference initialized by S2 refers is more cv-qualified than the type to which the reference initialized by S1 refers where T1 and T2 are not the same type, and T2 is reference-compatible with T1. [Example 6: ...

```
int h1(int (&)[1]);
int h1(int (&)[1]);
int h2(void (&)());
int h2(void (&)() noexcept);
void g2() {
    int a[1];
    h1(a); // calls h1(int (&)[1])
    extern void f2() noexcept;
    h2(f2); // calls h2(void (&)() noexcept)
}
```

-- end example]

2809. An implicit definition does not redeclare a function

Section: 9.5.2 [dcl.fct.def.default] Status: ready Submitter: Brian Bi Date: 2023-09-27

Subclause 9.5.2 [dcl.fct.def.default] paragraph 5 specifies:

... A user-provided explicitly-defaulted function (i.e., explicitly defaulted after its first declaration) is implicitly defined at the point where it is explicitly defaulted; if such a function is implicitly defined as deleted, the program is ill-formed. A non-user-provided defaulted function (i.e., implicitly declared or explicitly defaulted in the class) that is not defined as deleted is implicitly defined when it is odr-used (6.3 [basic.def.odr]) or needed for constant evaluation (7.7 [expr.const]).

In the first case, there is a second point of declaration for the function, wherever the user wrote the definition. In contrast, there is no redeclaration for the second case, where the function is not user-provided. A note would clarify.

Proposed resolution (approved by CWG 2024-03-20):

Insert a paragraph break before the quoted section and change in 9.5.2 [dcl.fct.def.default] paragraph 5 as follows:

... A function is *user-provided* if it is user-declared and not explicitly defaulted or deleted on its first declaration. A user-provided explicitly-defaulted function (i.e., explicitly defaulted after its first declaration) is implicitly defined at the point where it is explicitly defaulted; if such a function is implicitly defined as deleted, the program is ill-formed. [Note 1: Declaring a function as defaulted after its first declaration can provide efficient execution and concise definition while enabling a stable binary interface to an evolving code base. —end note] A non-user-provided defaulted function (i.e., implicitly declared or explicitly defaulted in the class) that is not defined as deleted is implicitly defined when it is odr-used (6.3 [basic.def.odr]) or needed for constant evaluation (7.7 [expr.const]). [Note 1: Declaring a function as defaulted after its first declaration can provide efficient execution and concise definition while enabling a stable binary interface to an evolving code base. —end note] [Note: The implicit definition of a non-user-provided defaulted function does not bind any names. —end note]

2810. Requiring the absence of diagnostics for templates

Section: 13.8.1 [temp.res.general] Status: ready Submitter: Andrey Erokhin Date: 2022-01-23

Subclause 13.8.1 [temp.res.general] paragraph 6 specifies:

The program is ill-formed, no diagnostic required, if: ... Otherwise, no diagnostic shall be issued for a template for which a valid specialization can be generated.

The "Otherwise..." part is misleading; it could be interpreted to mean that warnings must be suppressed in templates.

Proposed resolution (approved by CWG 2023-12-01):

Change in 13.8.1 [\[temp.res.general\]](#) paragraph 6 as follows:

~~Otherwise, no diagnostic shall be issued for a template for which a valid specialization can be generated.~~

2811. Clarify "use" of main

Section: 6.9.3.1 [\[basic.start.main\]](#) **Status:** ready **Submitter:** Jan Schultke **Date:** 2023-10-12

Subclause 6.9.3.1 [\[basic.start.main\]](#) paragraph 3 specifies:

The function main shall not be used within a program. ...

It is unclear what "use" means. N3214 excluded this appearance from the clarifications of "use" that were turned into "odr-use". For example, it is unclear whether `decltype(main)` is allowed or not.

CWG 2023-12-01

CWG favored to ban any mention of main.

Proposed resolution (approved by CWG 2023-12-15):

Change in 6.9.3.1 [\[basic.start.main\]](#) paragraph 3 as follows:

The function main shall not be ~~used within a program~~ **named by an expression**. ...

2813. Class member access with prvalues

Section: 7.6.1.5 [\[expr.ref\]](#) **Status:** ready **Submitter:** Barry Revzin **Date:** 2023-08-28

Subclause 7.6.1.5 [\[expr.ref\]](#) paragraph 2 specifies:

For the first option (dot) the first expression shall be a glvalue. ...

This provision forces a temporary materialization conversion (i.e. a copy) per 7.2.1 [\[basic.lval\]](#) paragraph 7:

Whenever a prvalue appears as an operand of an operator that expects a glvalue for that operand, the temporary materialization conversion (7.3.5 [\[conv.rval\]](#)) is applied to convert the expression to an xvalue.

However, this is limiting in the case that an explicit object member function with a by-value object parameter is invoked for a non-copyable class object, for example:

```
struct X {
    X() = default;

    X(const X&) = delete;
    X& operator=(const X&) = delete;

    void f(this X self) { }
};

void f() {
    X{}.f();    // OK?
}
```

The example ought to be well-formed.

Proposed resolution (reviewed by CWG 2024-02-16) [SUPERSEDED]:

1. Change in 6.7.7 [\[class.temporary\]](#) paragraph 2 as follows:

- ... *[Note 3: Temporary objects are materialized:*
- when binding a reference to a prvalue (9.4.4 [\[dcl.init.ref\]](#), 7.6.1.4 [\[expr.type.conv\]](#), 7.6.1.7 [\[expr.dynamic.cast\]](#), 7.6.1.9 [\[expr.static.cast\]](#), 7.6.1.11 [\[expr.const.cast\]](#), 7.6.3 [\[expr.cast\]](#)),
 - when performing **certain** member ~~access~~ **accesses** on a class prvalue (7.6.1.5 [\[expr.ref\]](#), 7.6.4 [\[expr.mptr.oper\]](#)),
 - when invoking an implicit object member function on a class prvalue (7.6.1.3 [\[expr.call\]](#)),
 - ...
- end note]

2. Change in 7.6.1.3 [\[expr.call\]](#) paragraph 6 as follows:

When a function is called, each parameter (9.3.4.6 [del.fct]) is initialized (9.4 [del.init], 11.4.5.3 [class.copy.ctor]) with its corresponding argument. If the function is an explicit object member function and there is an implied object argument (12.2.2.2.2 [over.call.func]), the list of provided arguments is preceded by the implied object argument for the purposes of this correspondence. If there is no corresponding argument, the default argument for the parameter is used. [Example 1:

```
template<typename ...T> int f(int n = 0, T ...t);
int x = f<int>(); // error: no argument for second function parameter
```

—end example] If the function is a static member function invoked using class member access syntax, the object expression is a discarded-value expression (7.2.3 [expr.context]). If the function is an implicit object member function, the object expression of the class member access shall be a glvalue and the this parameter of the function (7.5.2 [expr.prim.this]) is initialized with a pointer to the object of the call, converted as if by an explicit type conversion (7.6.3 [expr.cast]). [Note 5: There is no access or ambiguity checking on this conversion; the access checking and disambiguation are done as part of the (possibly implicit) class member access operator. See 6.5.2 [class.member.lookup], 11.8.3 [class.access.base], and 7.6.1.5 [expr.ref]. —end note] When a function is called, the type of any parameter shall not be a class type that is either incomplete or abstract. [Note 6: This still allows a parameter to be a pointer or reference to such a type. However, it prevents a passed-by-value parameter to have an incomplete or abstract class type. —end note] It is implementation-defined whether ...

3. Change in 7.6.1.5 [expr.ref] paragraph 2 as follows:

For the first option (dot), if the id-expression is a data member, the first expression shall be a glvalue. For the second option (arrow), the first expression shall be a prvalue having pointer type. ...

CWG 2024-02-16

When a class member access refers to a static data member or a member enumerator, the object expression should also be treated as a discarded-value expression.

Proposed resolution (approved by CWG 2024-03-01):

1. Change in 6.7.7 [class.temporary] paragraph 2 as follows:

... [Note 3: Temporary objects are materialized:

- when binding a reference to a prvalue (9.4.4 [del.init.ref], 7.6.1.4 [expr.type.conv], 7.6.1.7 [expr.dynamic.cast], 7.6.1.9 [expr.static.cast], 7.6.1.11 [expr.const.cast], 7.6.3 [expr.cast]),
- when performing certain member accesses on a class prvalue (7.6.1.5 [expr.ref], 7.6.4 [expr.mptr.oper]),
- when invoking an implicit object member function on a class prvalue (7.6.1.3 [expr.call]),
- ...

—end note]

2. Change in 7.6.1.3 [expr.call] paragraph 6 as follows:

When a function is called, each parameter (9.3.4.6 [del.fct]) is initialized (9.4 [del.init], 11.4.5.3 [class.copy.ctor]) with its corresponding argument. If the function is an explicit object member function and there is an implied object argument (12.2.2.2.2 [over.call.func]), the list of provided arguments is preceded by the implied object argument for the purposes of this correspondence. If there is no corresponding argument, the default argument for the parameter is used. [Example 1:

```
template<typename ...T> int f(int n = 0, T ...t);
int x = f<int>(); // error: no argument for second function parameter
```

—end example] If the function is an implicit object member function, the object expression of the class member access shall be a glvalue and the this parameter of the function (7.5.2 [expr.prim.this]) is initialized with a pointer to the object of the call, converted as if by an explicit type conversion (7.6.3 [expr.cast]). [Note 5: There is no access or ambiguity checking on this conversion; the access checking and disambiguation are done as part of the (possibly implicit) class member access operator. See 6.5.2 [class.member.lookup], 11.8.3 [class.access.base], and 7.6.1.5 [expr.ref]. —end note] When a function is called, the type of any parameter shall not be a class type that is either incomplete or abstract. [Note 6: This still allows a parameter to be a pointer or reference to such a type. However, it prevents a passed-by-value parameter to have an incomplete or abstract class type. —end note] It is implementation-defined whether ...

3. Change in 7.6.1.5 [expr.ref] paragraph 2 as follows:

For the first option (dot), if the id-expression names a static member or an enumerator, the first expression is a discarded-value expression (7.2.3 [expr.context]); if the id-expression names a non-static data member, the first expression shall be a glvalue. For the second option (arrow), the first expression shall be a prvalue having pointer type. ...

2820. Value-initialization and default constructors

Section: 9.4.1 [del.init.general] Status: ready Submitter: Shafik Yaghmour Date: 2023-10-31

Subclause 9.4.1 [del.init.general] paragraph 9 specifies:

To value-initialize an object of type T means:

- if T is a (possibly cv-qualified) class type (Clause 11 [class]), then
 - if T has either no default constructor (11.4.5.2 [class.default.ctor]) or a default constructor that is user-provided or deleted, then the object is default-initialized;
 - otherwise, the object is zero-initialized and the semantic constraints for default-initialization are checked, and if T has a non-trivial default constructor, the object is default-initialized;
- if T is an array type, ...

The specification about checking the semantic constraints and invoking only non-trivial default constructors is overly convoluted. Omitting a call to a trivial constructor is an as-if optimization that should not be prescribed by the standard.

Proposed resolution (approved by CWG 2024-01-19):

Change in 9.4.1 [dcl.init.general] bullet 9.1.2 as follows:

- ...
- otherwise, the object is zero-initialized and ~~the semantic constraints for default initialization are checked, and if T has a non-trivial default constructor, the object is~~ **then** default-initialized;

2822. Side-effect-free pointer zap

Section: 6.7.5.1 [basic.stc.general] **Status:** ready **Submitter:** Davis Herring **Date:** 2023-11-06

Subclause 6.7.5.1 [basic.stc.general] paragraph 4 seems to suggest that the end of duration of a region of storage causes actual modifications to pointer objects, causing questions about data races (in the abstract machine).

Proposed resolution (approved by CWG 2024-03-20):

1. Append to 6.7.5.1 [basic.stc.general] paragraph 1:

[Note: After the duration of a region of storage has ended, the use of pointers to that region of storage is limited (6.8.4 [basic.compound]). -- end note]

2. Remove 6.7.5.1 [basic.stc.general] paragraph 4 as follows:

~~When the end of the duration of a region of storage is reached, the values of all pointers representing the address of any part of that region of storage become invalid pointer values (6.8.4 [basic.compound]). Indirection through an invalid pointer value and passing an invalid pointer value to a deallocation function have undefined behavior. Any other use of an invalid pointer value has implementation-defined behavior. [Footnote: ...]~~

3. Change in 6.8.4 [basic.compound] paragraph 3 as follows:

[Note 2: A pointer past the end of an object (7.6.6 [expr.add]) is not considered to point to an unrelated object of the object's type, even if the unrelated object is located at that address. ~~A pointer value becomes invalid when the storage it denotes reaches the end of its storage duration; see 6.7.5 [basic.stc].~~ —end note]

4. Insert a new paragraph after 6.8.4 [basic.compound] paragraph 3:

A pointer value *P* is *valid in the context of* an evaluation *E* if *P* is a null pointer value, or if it is a pointer to or past the end of an object *O* and *E* happens before the end of the duration of the region of storage for *O*. If a pointer value *P* is used in an evaluation *E* and *P* is not valid in the context of *E*, then the behavior is undefined if *E* is an indirection (7.6.2.2 [expr.unary.op]) or an invocation of a deallocation function (6.7.5.3 [basic.stc.dynamic.deallocation]), and implementation-defined otherwise. [Footnote: Some implementations might define that copying such a pointer value causes a system-generated runtime fault. -- end footnote] [Note: *P* can be valid in the context of *E* even if it points to a type unrelated to that of *O* or if *O* is not within its lifetime, although further restrictions apply to such pointer values (6.7.3 [basic.life], 7.2.1 [basic.lval], 7.6.6 [expr.add]). —end note]

5. Change in 7.6.1.9 [expr.static.cast] paragraph 14 as follows:

... If the original pointer value represents the address *A* of a byte in memory and *A* does not satisfy the alignment requirement of *T*, then the resulting pointer value (6.8.4 [basic.compound]) is unspecified. ...

6. Change in 7.6.1.10 [expr.reinterpret.cast] paragraph 5 as follows:

A value of integral type or enumeration type can be explicitly converted to a pointer. A pointer converted to an integer of sufficient size (if any such exists on the implementation) and back to the same pointer type will have its original value (6.8.4 [basic.compound]); mappings between pointers and integers are otherwise implementation-defined.

2824. Copy-initialization of arrays

Section: 9.4.1 [dcl.init.general] **Status:** ready **Submitter:** Anoop Rana **Date:** 2023-11-06

Consider:

```
std::string arr[] = "some string";
```

Prior to the application of paper P0960R3 (Allow initializing aggregates from a parenthesized list of values), this was ill-formed, but now it is well-formed. However, the specification talks about an *expression-list* as part of the initializer, which does not exist in this case.

Proposed resolution (approved by CWG 2023-11-07):

Change in 9.4.1 [[dcl.init.general](#)] bullet 16.5 as follows:

Otherwise, if the destination type is an array, the object is initialized as follows. **The initializer shall be of the form (*expression-list*).** Let $x_1, \dots, x_k$ be the elements of the *expression-list*. If the destination type is an array of unknown bound, it is defined as having k elements. Let n denote the array size after this potential adjustment. If k is greater than n , the program is ill-formed. Otherwise, ...

2825. Range-based for statement using a *braced-init-list*

Section: 8.6.5 [[stmt.ranged](#)] **Status:** ready **Submitter:** Arthur O'Dwyer **Date:** 2023-11-08

Consider:

```
for (int i : { 1, 2, 3 }) // argument-dependent lookup for begin(std::initializer_list<int>)
/* ... */;
```

This is undesirable; instead, a member function `begin` should be preferred.

Proposed resolution (approved by CWG 2023-11-09):

Change in 8.6.5 [[stmt.ranged](#)] bullet 1.3 as follows:

begin-expr and *end-expr* are determined as follows:

- if the ~~for-range-initializer is an expression of~~ **type of range is a reference to an** array type R , *begin-expr* and *end-expr* are `range` and `range + N`, respectively, where N is the array bound. If R is an array of incomplete type, the program is ill-formed;
- if the ~~for-range-initializer is an expression of~~ **type of range is a reference to a** class type C , and searches in the scope of C (6.5.2 [[class.member.lookup](#)]) for the names `begin` and `end` each find at least one declaration, *begin-expr* and *end-expr* are `range.begin()` and `range.end()`, respectively;
- otherwise, *begin-expr* and *end-expr* are `begin(range)` and `end(range)`, respectively, where `begin` and `end` undergo argument-dependent lookup (6.5.4 [[basic.lookup.argdep](#)]). [*Note 1*: Ordinary unqualified lookup (6.5.3 [[basic.lookup.unqual](#)]) is not performed. —end note]

Additional notes (November, 2023)

Forwarded to EWG via [paper issue 1694](#) for confirmation of the design direction.

EWG 2023-11-09

Approved by EWG.

2828. Ambiguous interpretation of C-style cast

Section: 7.6.3 [[expr.cast](#)] **Status:** ready **Submitter:** Jim X **Date:** 2022-03-21

(From editorial issue [5355](#).)

Consider:

```
int*** ptr = 0;
auto t = (int const*const*const*)ptr;
```

There is more than one way how this can be interpreted as a `static_cast` followed by a `const_cast`, namely:

```
const_cast<int const * const * const *>(static_cast<int * * const *>(ptr));
const_cast<int const * const * const *>(static_cast<int * const * const *>(ptr));
```

Subclause 7.6.3 [[expr.cast](#)] paragraph 4 makes such a program ill-formed:

... If a conversion can be interpreted in more than one way as a `static_cast` followed by a `const_cast`, the conversion is ill-formed.

Proposed resolution (approved by CWG 2024-03-20):

Change in 7.6.3 [[expr.cast](#)] paragraph 4 as follows:

... If a conversion can be interpreted in more than one of the ways listed above, the interpretation that appears first in the list is used, even if a cast resulting from that interpretation is ill-formed. If a ~~conversion can be interpreted in more than one way as a~~ **is used and the conversion can be interpreted in more than one way as such**, the conversion is ill-formed. [Example 1 :

```
struct A { };
struct I1 : A { };
struct I2 : A { };
struct D : I1, I2 { };
A* foo( D* p ) {
    return (A*)( p ); // ill-formed static_cast interpretation
}
```

```
int*** ptr = 0;
auto t = (int const*const*const*)ptr; // OK, const_cast interpretation

struct S {
    operator const int*();
    operator volatile int*();
};
int *p = (int*)S(); // error: two possible interpretations using static_cast followed by const_cast

-- end example ]
```

2830. Top-level cv-qualification should be ignored for list-initialization

Section: 9.4.5 [[dcl.init.list](#)] **Status:** ready **Submitter:** Krystian Stasiowski **Date:** 2019-11-19

(From editorial issue [3492](#).)

Subclause 9.4.5 [[dcl.init.list](#)] paragraph 3 specifies:

List-initialization of an object or reference of type T is defined as follows:

- ...

Top-level cv-qualifiers should be ignored when comparing T to other types, e.g. in 9.4.5 [[dcl.init.list](#)] bullet 3.2.

Proposed resolution (approved by CWG 2024-01-19):

Change in 9.4.5 [[dcl.init.list](#)] paragraph 3 as follows:

List-initialization of an object or reference of type **cv** T is defined as follows:

- ...
- If T is an aggregate class and the initializer list has a single element of type ~~cv~~ **cv** U, where U is T or a class derived from T, the object is initialized from that element (by copy-initialization for copy-list-initialization, or by direct-initialization for direct-list-initialization).
- ...

2831. Non-templated function definitions and *requires*-clauses

Section: 9.3.1 [[dcl.decl.general](#)] **Status:** ready **Submitter:** Krystian Stasiowski **Date:** 2020-04-13

(From editorial issue [3936](#).)

Subclause 9.3.1 [[dcl.decl.general](#)] paragraph 4 specifies:

The optional *requires*-clause in an *init-declarator* or *member-declarator* shall be present only if the declarator declares a templated function (13.1 [[temp.pre](#)]). ...

This rule does not address function definitions, because those have neither an *init-declarator* nor a *member-declarator*.

Proposed resolution (approved by CWG 2024-03-20):

Change in 9.5.1 [[dcl.fct.def.general](#)] paragraph 1 as follows:

... The optional *attribute-specifier-seq* in a *function-definition* appertains to the function. A *virt-specifier-seq* can be part of a *function-definition* only if it is **function-definition with a virt-specifier-seq shall be** a *member-declaration* (11.4 [[class.mem](#)]). **A function-definition with a *requires*-clause shall define a templated function.**

2845. Make the closure type of a captureless lambda a structural type

Section: 7.5.5.2 [[expr.prim.lambda.closure](#)] **Status:** ready **Submitter:** Barry Revzin **Date:** 2023-12-29

Issue 2542 (approved in June, 2023) made all closure types not be structural types, i.e. unsuitable for use as non-type template parameters. This causes an inconsistency with the treatment of the pointer-to-function conversion for closure types with no captures:

```
template <auto V>
void foo() {}

void bar() {
    foo<[i = 3] { return i; }>(); // #1: error
```



```
foo<[]{}>(); // #2: error
foo<+[]{}>(); // #3: OK, a function pointer is a structural type
}
```

Proposed resolution (approved by CWG 2024-02-02):

1. Change in 7.5.5.2 [expr.prim.lambda.closure] paragraph 3 as follows:

The closure type is not an aggregate type (9.4.2 [dcl.init.aggr]) ~~and not~~; it is a structural type (13.2 [temp.param]) **if and only if the lambda has no lambda-capture**. An implementation may define the closure type differently from ...

2. Change in 13.6 [temp.type] paragraph 2 as follows:

Two values are *template-argument-equivalent* if they are of the same type and

- ...
- they are of a closure type (7.5.5.2 [expr.prim.lambda.closure]), or
- they are of class type and their corresponding direct subobjects and reference members are template-argument-equivalent.

2846. Out-of-class definitions of explicit object member functions

Section: 9.3.4.6 [dcl.fct] **Status:** ready **Submitter:** Krystian Stasiowski **Date:** 2024-01-28

(From submission #493.)

Consider:

```
struct A {
    void f(this A&);
};
void A::f(this A&) { } // #1
```

This is accepted by all major implementations. However, 9.3.4.6 [dcl.fct] paragraph 6 specifies:

An *explicit-object-parameter-declaration* is a *parameter-declaration* with a *this* specifier. An explicit-object-parameter-declaration shall appear only as the first *parameter-declaration* of a *parameter-declaration-list* of either:

- a *member-declarator* that declares a member function (11.4 [class.mem]), or
- a *lambda-declarator* (7.5.5 [expr.prim.lambda]).

A *member-declarator* with an explicit-object-parameter-declaration shall not include a *ref-qualifier* or a *cv-qualifier-seq* and shall not be declared static or virtual.

The *function-definition* at #1 is neither of the two allowed options.

Similar concerns arise for explicit instantiations and explicit specializations.

Proposed resolution (approved by CWG 2024-02-16):

Change in 9.3.4.6 [dcl.fct] paragraph 6 as follows:

An *explicit-object-parameter-declaration* is a *parameter-declaration* with a *this* specifier. An explicit-object-parameter-declaration shall appear only as the first *parameter-declaration* of a *parameter-declaration-list* of ~~either~~ **one of**:

- a ~~member-declarator that declares~~ **declaration of** a member function **or member function template** (11.4 [class.mem]), or
- an explicit instantiation (13.9.3 [temp.explicit]) or explicit specialization (13.9.4 [temp.expl.spec]) of a templated member function, or
- a *lambda-declarator* (7.5.5 [expr.prim.lambda]).

A *member-declarator* with an explicit-object-parameter-declaration shall not include a *ref-qualifier* or a *cv-qualifier-seq* and shall not be declared static or virtual.

2848. Omitting an empty template argument list for explicit instantiation

Section: 13.9.3 [temp.explicit] **Status:** ready **Submitter:** Anoop Rana **Date:** 2024-01-15

(From submission #489.)

Subclause 13.9.3 [temp.explicit] paragraph 8 specifies:

A trailing *template-argument* can be left unspecified in an explicit instantiation of a function template specialization or of a member function template specialization provided it can be deduced (13.10.3.7 [temp.deduct.decl]). If all template arguments can be deduced, the empty template argument list $\diamond$ may be omitted. [Example 3:

```
template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v) { /* ... */ }

// instantiate sort(Array<int>&) -- template-argument deduced
template void sort<>(Array<int>&);

—end example]
```

This paragraph is redundant with a more general provision on explicitly specifying template arguments in 13.10.2 [temp.arg.explicit] paragraph 4:

Trailing template arguments that can be deduced (13.10.3 [temp.deduct]) or obtained from default *template-arguments* may be omitted from the list of explicit *template-arguments*. [Note 1: A trailing template parameter pack (13.7.4 [temp.variadic]) not otherwise deduced will be deduced as an empty sequence of template arguments. —end note] If all of the template arguments can be deduced or obtained from default *template-arguments*, they may all be omitted; in this case, the empty template argument list <> itself may also be omitted. ...

A similar duplication is in 13.9.4 [temp.expl.spec] paragraph 10.

Proposed resolution (approved by CWG 2024-02-16):

1. Change the example in 13.9.3 [temp.explicit] paragraph 4 as follows:

```
...
template<class T> void sort(Array<T>& v) { /* ... */ }
template void sort(Array<char>&); // argument is deduced here (13.10.2 [temp.arg.explicit])
...
```

2. Remove 13.9.3 [temp.explicit] paragraph 8:

A trailing *template-argument* can be left unspecified in an explicit instantiation of a function template specialization or of a member function template specialization provided it can be deduced (13.10.3.7 [temp.deduct.decl]). If all template arguments can be deduced, the empty template argument list <> may be omitted. [Example 3:

```
template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v) { /* ... */ }

// instantiate sort(Array<int>&) -- template-argument deduced
template void sort<>(Array<int>&);

—end example]
```

3. Change the example in 13.9.4 [temp.expl.spec] paragraph 1 as follows:

... [Example:

```
template<class T> class stream;
template<> class stream<char> { /* ... */ }; // #1

template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v) { /* ... */ }

template<> void sort<int>(Array<int>&); // #2
template<> void sort<char*>(Array<char*>&); // #3 template argument is deduced (13.10.2 [temp.arg.explicit])
...
```

Given these declarations, `stream<char>` #1 will be used as the definition of streams of chars; other streams will be handled by class template specializations instantiated from the class template. Similarly, #2 will be used as the sort function for arguments of type `Array<int>` and #3 `sort<char*>` will be used as the sort function for arguments of type `Array<char*>`; other `Array` types will be sorted by functions generated from the `function` template. —end example]

4. Remove 13.9.4 [temp.expl.spec] paragraph 10:

A trailing *template-argument* can be left unspecified in the *template-id* naming an explicit function template specialization provided it can be deduced (13.10.3.7 [temp.deduct.decl]). [Example 6:

```
template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v);

// explicit specialization for sort(Array<int>&)
// with deduced template-argument of type int
template<> void sort(Array<int>&);

—end example]
```

2849. Parameter objects are not temporary objects

Section: 6.7.7 [class.temporary] Status: ready Submitter: Brian Bi Date: 2024-01-20

(From submission #490.)

Parameter objects are not temporary objects, according to 6.7.7 [class.temporary] paragraph 1. An exception hinting at this in 6.7.7 [class.temporary] paragraph 7 should be removed.

Proposed resolution (approved by CWG 2024-02-02):

Change in 6.7.7 [class.temporary] paragraph 7 as follows:

The fourth context is when a temporary object ~~other than a function parameter object~~ is created in the *for-range-initializer* of a range-based for statement. If such a temporary object would otherwise be destroyed at the end of the *for-range-initializer* full-expression, the object persists for the lifetime of the reference initialized by the *for-range-initializer*.

2850. Unclear storage duration for function parameter objects

Section: 6.7.5 [basic.stc] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-02-03

(From submission #490.)

Function parameter objects have automatic storage duration and are not temporary objects (see also issue 2849). However, it is unclear how long the storage for function parameter objects lasts.

Furthermore, for temporary objects that are destroyed at the end of the full-expression, it is unclear how the destruction is ordered with respect to temporary objects destroyed at the end of the full-expression.

Proposed resolution (approved by CWG 2024-03-20):

1. Change in and combine 6.7.5.4 [basic.stc.auto] paragraph 1 and 2 as follows:

Variables that belong to a block ~~or parameter~~ scope and are not explicitly declared static, thread_local, or extern have automatic storage duration. The storage for ~~these entities~~ **such variables** lasts until the block in which they are created exits. [Note 1: These variables are initialized and destroyed as described in 8.8 [stmt.dcl]. -- end note]

Variables that belong to a parameter scope also have automatic storage duration. The storage for a function parameter lasts until immediately after its destruction (7.6.1.3 [expr.call]).

2. Change in 6.7.7 [class.temporary] paragraph 8 as follows:

~~The destruction of a temporary whose lifetime is not extended beyond the full-expression in which it was created is sequenced before the destruction of every temporary which is constructed earlier in the same full-expression.~~ **Let x and y each be either a temporary object whose lifetime is not extended, or a function parameter. If the lifetimes of x and y end at the end of the same full-expression, and x is initialized before y, then the destruction of y is sequenced before that of x.** If the lifetime of two or more temporaries with lifetimes extending beyond the full-expressions in which they were created ends at the same point, these temporaries are destroyed at that point in the reverse order of the completion of their construction. In addition, the destruction of such temporaries shall take into account the ordering of destruction of objects with static, thread, or automatic storage duration (6.7.5.2 [basic.stc.static], 6.7.5.3 [basic.stc.thread], 6.7.5.4 [basic.stc.auto]); that is, if obj1 is an object with the same storage duration as the temporary and created before the temporary is created the temporary shall be destroyed before obj1 is destroyed; if obj2 is an object with the same storage duration as the temporary and created after the temporary is created the temporary shall be destroyed after obj2 is destroyed.

3. Change in 7.6.1.3 [expr.call] paragraph 6 as follows:

... It is implementation-defined whether ~~the lifetime of a parameter ends~~ **is destroyed** when the function in which it is defined ~~returns~~ **exits** (8.7.4 [stmt.return], 14.3 [except.ctor]) or at the end of the enclosing full-expression; **parameters are always destroyed in the reverse order of their construction.** The initialization and destruction of each parameter occurs within the context of the full-expression (6.9.1 [intro.execution]) where the function call appears.

4. Change in 8.8 [stmt.dcl] paragraph 2 as follows:

A **block** variable with automatic storage duration (6.7.5.4 [basic.stc.auto]) is active everywhere in the scope to which it belongs after its *init-declarator*. Upon each transfer of control (including sequential execution of statements) within a function from point P to point Q, all **block** variables with automatic storage duration that are active at P and not at Q are destroyed in the reverse order of their construction. Then, all **block** variables with automatic storage duration that are active at Q but not at P are initialized in declaration order; unless all such variables have vacuous initialization (6.7.3 [basic.life]), the transfer of control shall not be a jump. [Footnote: ...] When a *declaration-statement* is executed, P and Q are the points immediately before and after it; when a function returns, Q is after its body.

2851. Allow floating-point conversions in converted constant expressions

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-11-03

(From submission #456.)

With the adoption of P1907R1, non-type template parameters of floating-point types have been introduced. It is surprising that a double value cannot be passed as a template argument for a template parameter of type long double, because floating-point conversions are not allowed in converted constant expressions.

Proposed resolution (approved by CWG 2024-02-16):

Change in 7.7 [expr.const] paragraph 12 as follows:

A *converted constant expression* of type T is an expression, implicitly converted to type T, where the converted expression is a constant expression and the implicit conversion sequence contains only

- user-defined conversions,
- lvalue-to-rvalue conversions (7.3.2 [conv.lval]),
- array-to-pointer conversions (7.3.3 [conv.array]),
- function-to-pointer conversions (7.3.4 [conv.func]),
- qualification conversions (7.3.6 [conv.qual]),
- integral promotions (7.3.7 [conv.prom]),
- integral conversions (7.3.9 [conv.integral]) other than narrowing conversions (9.4.5 [decl.init.list]),
- floating-point promotions (7.3.8 [conv.fpprom]),
- floating-point conversions (7.3.10 [conv.double]) where the source value can be represented exactly in the destination type,
- null pointer conversions (7.3.12 [conv.ptr]) from std::nullptr_t,
- null member pointer conversions (7.3.13 [conv.mem]) from std::nullptr_t, and
- function pointer conversions (7.3.14 [conv.fctptr]),

and where the reference binding (if any) binds directly.

2853. Pointer arithmetic with pointer to hypothetical element

Section: 7.6.6 [expr.add] **Status:** ready **Submitter:** Jim X **Date:** 2024-02-03

(From submission #495.)

The phrasing in 7.6.6 [expr.add] bullet 4.2 excludes pointer arithmetic on a pointer pointing to the hypothetical (past-the-end) array element.

Proposed resolution (approved by CWG 2024-02-16):

Change in 7.6.6 [expr.add] bullet 4.2 as follows:

- ...
- Otherwise, if P points to ~~an~~ **a (possibly-hypothetical)** array element i of an array object x with n elements (9.3.4.5 [decl.array]), [Footnote: ...] the expressions P + J and J + P (where J has the value j) point to the (possibly-hypothetical) array element i + j of x if 0 <= i + j <= n and the expression P - J points to the (possibly-hypothetical) array element i - j of x if 0 <= i - j <= n.

2854. Storage duration of exception objects

Section: 14.2 [except.throw] **Status:** ready **Submitter:** Jiang An **Date:** 2024-01-22

(From submission #492.)

According to 14.2 [except.throw] paragraph 3, the exception object is a temporary object. But none of the possible storage durations of temporary objects matches the behavior of exception objects.

Proposed resolution (approved by CWG 2024-02-16):

1. Change in 6.7.7 [class.temporary] paragraph 1 as follows:

Temporary objects are created

- when a prvalue is converted to an xvalue (7.3.5 [conv.rval]); **and**
- when needed by the implementation to pass or return an object of trivially copyable type (see below); ~~and,~~
- ~~when throwing an exception (14.2 [except.throw]).~~ **[Note 1: The lifetime of exception objects is described in 14.2 [except.throw].** ~~end note]~~

...

2. Change in 14.2 [except.throw] paragraph 3 as follows:

Throwing an exception initializes ~~a temporary~~ **an** object **with dynamic storage duration**, called the exception object. If the type of the exception object would be an incomplete type (6.8.1 [basic.types.general]), an abstract class type (11.7.4 [class.abstract]), or a pointer to an incomplete type other than cv void (6.8.4 [basic.compound]) the program is ill-formed.

2855. Undefined behavior in postfix increment

Section: 7.6.1.6 [expr.post.incr] **Status:** ready **Submitter:** Lénárd Szolnoki **Date:** 2023-12-12

(From submission [#479](#).)

Consider:

```
int8_t x = 127;
x++;
```

This has undefined behavior, because the resulting value is not representable as an `int8_t`. In contrast,

```
int8_t x = 127;
++x;
```

is well-defined, because it is equivalent to `x += 1`, which is equivalent to `x = (int)x + 1` after the usual arithmetic conversions (7.4 [\[expr.arith.conv\]](#)). No arithmetic overflow occurs. The presence or absence of undefined behavior is detectable in constant evaluation.

Proposed resolution (approved by CWG 2024-02-16):

1. Change in 7.6.1.6 [\[expr.post.incr\]](#) paragraph 1 as follows:

The value of a postfix `++` expression is the value of its operand. [Note 1: The value obtained is a copy of the original value. —end note] The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type other than `cv bool`, or a pointer to a complete object type. An operand with volatile-qualified type is deprecated; see D.4 [\[depr.volatile.type\]](#). The value of the operand object is modified (3.1 [\[defs.access\]](#)) by adding 1 to it as if it were the operand of the prefix `++` operator (7.6.2.3 [\[expr.pre.incr\]](#)). The value computation of the `++` expression is sequenced before the modification of the operand object. With respect to an indeterminately-sequenced function call, the operation of postfix `++` is a single evaluation. [Note 2: Therefore, a function call cannot intervene between the lvalue-to-rvalue conversion and the side effect associated with any single postfix `++` operator. —end note] The result is a prvalue. The type of the result is the cv-unqualified version of the type of the operand. If the operand is a bit-field that cannot represent the incremented value, the resulting value of the bit-field is implementation-defined. See also 7.6.6 [\[expr.add\]](#) and 7.6.19 [\[expr.ass\]](#).

2. Change in 7.6.2.3 [\[expr.pre.incr\]](#) as follows:

The operand of prefix `++` or `--` is modified (3.1 [\[defs.access\]](#)) by adding 1. The operand shall be a modifiable lvalue. The type of the operand shall not be an arithmetic type other than of type `cv bool`, or a pointer to a completely defined object type. An operand with volatile-qualified type is deprecated; see D.4 [\[depr.volatile.type\]](#). The result is the updated operand; it is an lvalue, and it is a bit-field if the operand is a bit-field. The expression `++x` is otherwise equivalent to `x+=1` and the expression `--x` is otherwise equivalent to `x-=1`. [Note 1: See the discussions of addition (7.6.6 [\[expr.add\]](#)) and assignment operators (7.6.19 [\[expr.ass\]](#)) for information on conversions. —end note]

The operand of prefix `--` is modified (3.1 [\[defs.access\]](#)) by subtracting 1. The requirements on the operand of prefix `--` and the properties of its result are otherwise the same as those of prefix `++`. [Note 2: For postfix increment and decrement, see 7.6.1.6 [\[expr.post.incr\]](#). —end note]

2856. Copy-list-initialization with explicit default constructors

Section: 12.2.2.8 [\[over.match.list\]](#) Status: ready Submitter: Anoop Rana Date: 2024-01-09

(From submission [#486](#).)

Consider:

```
struct A {
    explicit A(int = 10);
    A() = default;    // converting constructor (11.4.8.2 [class.conv.ctor] paragraph 1)
};

A a = {};           // #1, copy-initialization

int f(A);
int x = f({});      // #2

A b;                // #3
```

#1 and #2 are accepted by MSVC and EDG, but considered ambiguous by gcc and clang. #3 is rejected as ambiguous by all major implementations.

#1 is copy-list-initialization (9.4.5 [\[dcl.init.list\]](#) paragraph 1), and A has a default constructor, thus a is value-initialized (9.4.5 [\[dcl.init.list\]](#) bullet 3.4). The default constructors are user-provided, thus a is default-initialized (9.4.1 [\[dcl.init.general\]](#) bullet 8.1, 9.4.1 [\[dcl.init.general\]](#) bullet 7.1). Overload resolution then chooses a constructor according to 12.2.2.4 [\[over.match.ctor\]](#) paragraph 1:

... For copy-initialization (including default initialization in the context of copy-initialization), the candidate functions are all the converting constructors (11.4.8.2 [\[class.conv.ctor\]](#)) of that class. ...

Thus, the explicit constructor is not a candidate; #1 chooses the converting constructor.

In contrast, #2 uses the special rules for forming a list-initialization sequence (12.2.4.2.6 [\[over.ics.list\]](#) paragraph 7), which perform overload resolution according to 12.2.2.8 [\[over.match.list\]](#) paragraph 1, which considers all constructors for overload resolution (and makes the program ill-formed if an explicit constructor is chosen). For the example, overload resolution is ambiguous.

#3 performs default-initialization, and overload resolution then chooses a constructor according to 12.2.2.4 [\[over.match.ctor\]](#) paragraph 1:

... For direct-initialization or default-initialization that is not in the context of copy-initialization, the candidate functions are all the constructors of the class of the object being initialized. ...

In this case, the overload resolution is ambiguous.

Proposed resolution (approved by CWG 2024-02-16):

Change in 12.2.2.4 [over.match.ctor] paragraph 1 as follows:

When objects of class type are direct-initialized (9.4 [dcl.init]), copy-initialized from an expression of the same or a derived class type (9.4 [dcl.init]), or default-initialized (9.4 [dcl.init]), overload resolution selects the constructor. For direct-initialization or default-initialization ~~that is not in the context of copy-initialization~~ **(including default-initialization in the context of copy-list-initialization)**, the candidate functions are all the constructors of the class of the object being initialized. ~~For copy-initialization (including default initialization in the context of copy-initialization)~~ **Otherwise**, the candidate functions are all the converting constructors (11.4.8.2 [class.conv.ctor]) of that class. The argument list is the *expression-list* or *assignment-expression* of the initializer. **For default-initialization in the context of copy-list-initialization, if an explicit constructor is chosen, the initialization is ill-formed.**

CWG 2024-02-16

The fact that #2 considers all constructors was discussed (and established) in the C++11 timeframe when brace-initialization was first introduced. #1 should be consistent with that, even though the = is usually a clear sign that explicit constructors are not considered.

2857. Argument-dependent lookup with incomplete class types

Section: 6.5.4 [basic.lookup.argdep] **Status:** ready **Submitter:** Lewis Baker **Date:** 2024-02-08

Subclause 6.5.4 [basic.lookup.argdep] bullet 3.2 specifies:

- ...
- If T is a class type (including unions), its associated entities are: the class itself; the class of which it is a member, if any; and its direct and indirect base classes. ...
- ...

It is unclear what happens if T is incomplete, for example because it was instantiated from a template whose definition was not (yet) available.

Proposed resolution (approved by CWG 2024-03-01)

Change in 6.5.4 [basic.lookup.argdep] bullet 3.2 as follows:

- ...
- If T is a class type (including unions), its associated entities are: the class itself; the class of which it is a member, if any; and, **if it is a complete type**, its direct and indirect base classes. ...
- ...